

Data Management

Master degree of Engineering in Computer Science and Artificial Intelligence

Matteo Giamberardini, Ilaria Casolino, Francesco Innocenzi

a.a. 2025/2026

Contents

1	Data Warehousing	5
1.1	Data Warehouse	6
1.2	Query Types	8
1.3	Data Warehouse Architectures	9
1.3.1	Single-Layer Architecture	9
1.3.2	Two-Layer Architecture	10
1.4	The ETL Process (Extract, Transform, Load)	11
1.5	The Multidimensional Data Model and OLAP	14
1.6	Accessing the Data Warehouse	19
1.7	Implementing the Multidimensional Data Model	21
1.7.1	ROLAP (Relational OLAP)	21
1.7.2	MOLAP (Multidimensional OLAP)	21
1.8	Design Methodology	25
1.8.1	Facts	25
1.9	The Dimensional Fact Model (DFM)	26
1.9.1	Naming Conventions	28
1.9.2	Descriptive and Cross-Dimensional Attributes	29
1.9.3	Optional Dimensions and Attributes	29
1.9.4	Coverage and Convergence	31
1.9.5	Additivity and Measure Types	34
1.9.6	Additive vs. Non-Additive Dimensions	35
1.9.7	Non-Aggregable Measures and Shorthand Notation	35
1.9.8	Special Types of Fact Schemata	36
1.9.9	Integrity Constraints	37
1.10	Logical Design	38
1.10.1	Star Schema	38
1.10.2	Snowflake Schema	42
1.11	Performance Optimization via Views	45
1.12	Implementing Materialized Views in Relational Schemata	48
1.13	Managing and Routing Aggregate Queries	50
1.14	The Logical Design Phase	51
1.15	Managing Shared Hierarchies in Logical Design	53
1.16	Managing Many-to-Many Relationships (Multiple Arcs)	55
1.17	Handling Degenerate and Junk Dimensions	57

Chapter 1

Data Warehousing

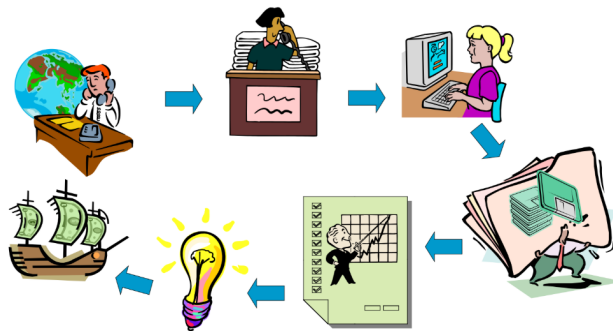
Enterprise Databases and the Shift to Business Intelligence

Until the mid-1980s, enterprise databases were mainly used to store **operational data**, that is, data produced by daily business activities. At that time, computer systems were considered simple support tools, useful for saving time and managing information more efficiently. They were not seen as tools for creating strategic value or improving business decisions.

Later, business needs changed. Companies started to require more advanced analysis to support decision-making. This led to the development of **Business Intelligence**. To support these analyses, organizations need a complete infrastructure, including hardware, networks, a reliable **Database Management System (DBMS)**, and both back-end and front-end software.

The goal of this infrastructure is to transform raw operational data into useful information that managers can analyze at different levels of detail.

The Data Warehouse Solution



Consider a large company with many branches. Managers may want to understand how each branch contributes to the overall performance. Instead of manually collecting reports from each location, the company uses a centralized repository.

This repository collects data from different sources, integrates it, and organizes it for analysis and decision-making. This centralized system is called a **Data Warehouse**.

A practical example is a university. Daily operations include managing classrooms, lectures, exams, and students. However, the university may also want to analyze long-term trends, such as:

- the number of enrolled students over several years,
- the average grades of courses,
- the number of graduates over time.

These analyses require historical data, which is exactly what a data warehouse provides.

OLTP (Online Transactional Processing) *Think of OLTP as cash registers.*

OLTP systems manage daily operations. For example, when a customer buys a product, the system records the transaction and updates the inventory immediately.

Their goal is to process many small and simple transactions very quickly, ensuring that business activities run smoothly.

OLAP (Online Analytical Processing) *Think of OLAP as business analysis.*

OLAP systems are used for complex analysis and decision-making. For example, an analyst may ask: “*How many products were sold across all stores last summer under different weather conditions?*”

OLAP systems analyze large amounts of historical data, usually stored in a data warehouse, to identify patterns and trends.

Separating OLAP and OLTP

If we try to run complex analytical queries on a system that is also handling daily transactions, the system may become very slow.

For example, if a cashier system is used to compute several years of sales analysis, it may stop working properly and delay customer service.

For this reason, OLTP and OLAP systems are separated:

- OLTP handles daily operations.
- OLAP uses a copy of the data in a data warehouse for analysis.

This separation improves performance and reliability.

1.1 Data Warehouse

A data warehouse is a collection of data, i.e. a central repository, that supports decision-making processes.

It provides the following features:

- **Subject-oriented**
- **Integrated and consistent**
- **Time-variant**, it shows its evolution over time
- **Non-volatile**

Subject-Oriented

Operational systems are organized around specific applications, such as reservations or transactions.

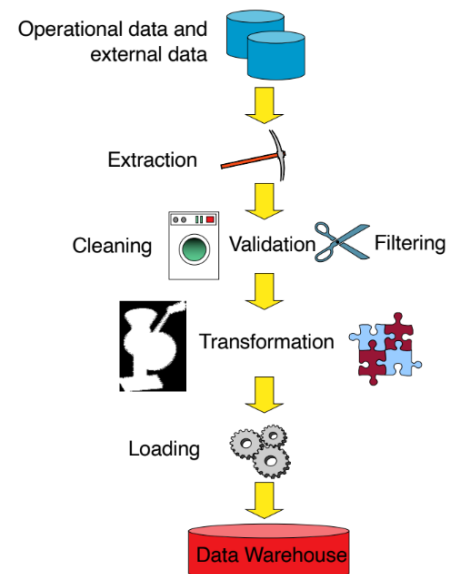
A data warehouse, instead, is organized around main business subjects, such as customers, products, or regions. This allows a more global and meaningful analysis.

Integrated and Consistent

A data warehouse combines data from different sources, such as operational databases or external systems.

To integrate data, a process is required:

1. **Extraction:** collecting data from sources.
2. **Cleaning and validation:** removing errors and improving quality.
3. **Transformation:** standardizing formats and structures.
4. **Loading:** storing data into the warehouse.



The final result is a unified and consistent view of company data. The goal is not to create new data, but to reorganize existing data in a useful way.

Shows Its Evolution Over Time

Operational databases usually store only current data. When a value changes, the old value is overwritten, and no history is kept.

A data warehouse, instead, stores historical data over long periods. It allows analysis over months or years. For this reason, time is always an important component of the data structure.

Non-Volatile

In operational systems, data is frequently updated, inserted, or deleted.

In a data warehouse, data is **non-volatile**. This means:

- data is not deleted,
- updates are performed periodically (often in batches) when data warehouses are offline,
- the system is mainly used for reading data, this means that data warehouses can be essentially viewed as read-only databases.

Because of this, transaction management is simpler than in OLTP systems. The main challenges are efficient query processing (*query-throughput*) and system reliability (*resilience*).

1.2 Query Types

The differences between OLTP and OLAP systems are also visible in the types of queries they execute.

- **OLTP (Online Transactional Processing):** Queries are simple and fast. They usually access a small number of tuples and are predefined inside applications. Ad-hoc queries are uncommon.
- **OLAP (Online Analytical Processing):** Queries are complex and flexible. They analyze large volumes of data and support interactive exploration. The workload changes frequently depending on user needs.

Summarizing the Differences

Feature	Operational Databases	Data Warehouses
Users	Thousands	Hundreds
Workload	Preset transactions	Specific analysis queries
Access	To hundreds of records, write and read mode	To millions of records, mainly read-only mode
Goal	Depends on applications	Decision-making support
Data	Detailed, both numeric and alphanumeric	Summed up, mainly numeric
Data integration	Application-based	Subject-based
Quality	In terms of integrity	In terms of consistency
Time coverage	Current data only	Current and historical data
Updates	Continuous	Periodical
Model	Normalized	Denormalized, multidimensional
Optimization	For OLTP access to a database part	For OLAP access to most of the database

1.3 Data Warehouse Architectures

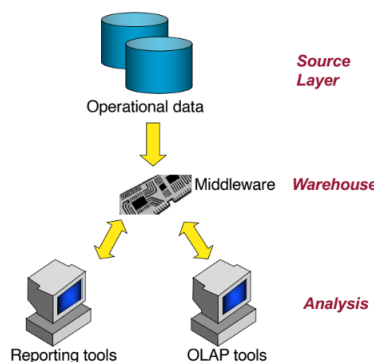
Architecture Requirements

When designing a data warehouse architecture, some important requirements must be satisfied to guarantee that the system works correctly and can evolve over time.

- **Separation:** Analytical processing and transactional processing should be kept separate to avoid interference between them.
- **Scalability:** The system must be easy to expand, both in hardware and software, so it can manage increasing data volumes and new user needs.
- **Extensibility:** The architecture should allow the integration of new applications and technologies without redesigning the entire system.
- **Security:** Since a data warehouse contains strategic data, it is important to control and monitor user access.
- **Administerability:** The system should be easy to manage and maintain on a daily basis.

1.3.1 Single-Layer Architecture

The goal of a single-layer architecture is to reduce the amount of stored data. In this case, only the operational data sources physically exist, and the data warehouse is *virtual*.



This solution has several disadvantages:

- It does not provide a proper separation between OLTP (operational) and OLAP (analytical) systems.
- Analytical queries are executed directly on operational databases (through middleware), which slows down daily transactions.
- It does not support the storage of historical data beyond what is already available in the sources.

For these reasons, single-layer architectures are rarely used today. They are only suitable when analytical requirements are very limited.

1.3.2 Two-Layer Architecture

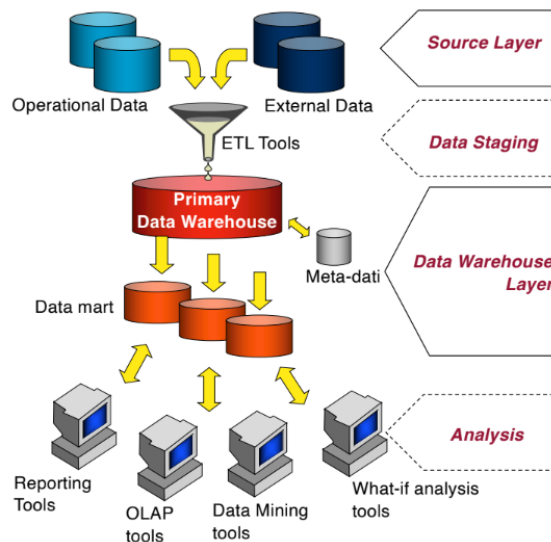
The two-layer architecture introduces a clear physical separation between operational data and the data warehouse.

Even if it is called “two-layer,” the data flow includes multiple steps: Source Layer, Data Staging (ETL), Data Warehouse Layer, Analysis.

An important concept in this architecture is the **Data Mart**. A data mart is a subset or aggregation of the data stored in a primary data warehouse. It includes a set of information relevant to a specific business area, corporate department, or category of users.

Types of Data Marts

- **Dependent Data Marts:** These are populated from a primary data warehouse. They are useful in medium and large organizations because they simplify development, provide targeted information to specific user groups, and improve query performance.
- **Independent Data Marts:** These are populated directly from operational sources, without using a central data warehouse. This simplifies the design, but may lead to inconsistencies between different data marts.



Advantages of the Two-Layer Architecture

- Data for analysis is available even if operational systems are temporarily offline.
- Analytical queries do not affect daily transactions.
- The data warehouse uses a multidimensional model optimized for analysis, while operational systems usually use relational or semi-structured models.

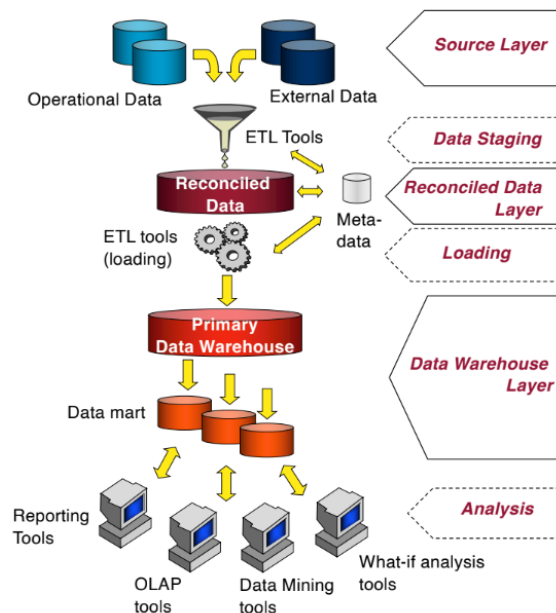
- It resolves the natural mismatch in time and granularity: OLTP systems manage current data at a maximum level of detail, while OLAP systems manage historical, summarized data.
- It allows optimization techniques specifically designed for analytical queries.

Three-Layer Architecture

The three-layer architecture extends the two-layer model by adding an intermediate layer called the **Reconciled Data Layer**.

The workflow includes: Source Layer, Data Staging, Reconciled Data Layer, Loading, Data Warehouse Layer, Analysis.

The **Reconciled Data Layer** stores data only after it has been cleaned and integrated. Therefore, data in this layer is consistent, correct, and detailed.



Advantages and Disadvantages of the Three-Layer Architecture

- **Main Advantage:** It creates a single and common reference model for the entire enterprise. It also separates the integration phase from the data warehouse loading phase.
- **Main Disadvantage:** It introduces redundancy because it requires storing an additional copy of detailed data before loading it into the warehouse.

1.4 The ETL Process (Extract, Transform, Load)

ETL processes are used to extract, integrate, and clean data coming from different operational sources, in order to load it into a data warehouse.

The main goal of ETL is to create a single, high-quality, and detailed data source. This process is called **Reconciliation**.

Depending on the architecture, the reconciled data can be:

- **Physical:** ETL processes load data into a physical reconciled data layer.
- **Virtual:** ETL processes load data directly into the data warehouse or data marts.

Reconciliation is performed both when the data warehouse is created for the first time and during each update.

The Four Phases of ETL and the Staging Area

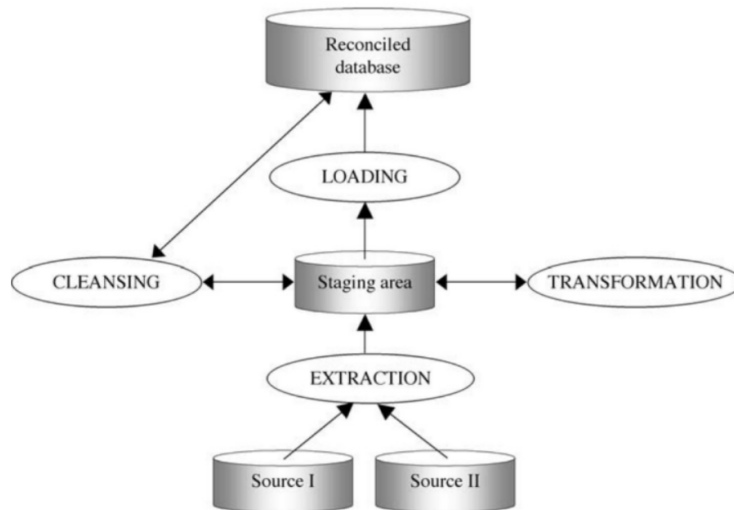
ETL stands for Extraction, Transformation, and Loading. However, in practice, the Transformation phase is divided into two parts:

- **Cleansing:** Correcting data *values*.
- **Transformation:** Correcting data *formats*.

For this reason, ETL is usually described as a four-phase process: Extraction, Cleansing, Transformation, Loading.

During these phases, data is stored in a temporary area called the **Staging Area**. This area contains raw data while cleansing and transformation are applied.

The loading phase may also be divided into two steps: first loading data into the reconciled layer, and then into the final data warehouse or data marts.



Phase 1: Extraction

Extraction is the process of collecting relevant data from source systems. There are two main approaches:

- **Static Extraction:** All data is extracted from the sources after a complete scan, creating a complete *snapshot*. This is typically used when the data warehouse is populated for the first time.
- **Incremental Extraction (Change Data Capture - CDC):** Only the changes made since the last extraction are collected. This method is used for regular updates.
 - *Immediate techniques (source-driven):* Changes are captured in real time using triggers, logs, or application support.
 - *Delayed techniques:* Changes are detected later using timestamps or by comparing new and old data.

In practice, incremental extraction tracks modifications using information such as the update date (e.g., `lcDate`) and the type of operation: I (Insert), U (Update), or D (Delete).

Phase 2: Cleansing

Data coming from operational systems is often of low quality. The goal of the Cleansing phase is to improve data quality by fixing common problems, such as:

- Duplicate records
- Inconsistent values (e.g., mismatch between city and ZIP code)
- Missing data fields
- Incorrect use of fields (e.g., putting a phone number in an address line)
- Invalid or impossible values
- Differences due to different conventions (e.g., using different abbreviations for the same word)
- Typing errors

Phase 3: Transformation

Transformation converts data from its original format into the format required by the data warehouse. This step is often complex because data may come from heterogeneous and unstructured sources.

Transformation rules depend on the target:

- **Reconciled Data Layer:** Includes *conversion and standardization* (uniform formats and units), *matching* (linking equivalent data from different sources), and *selection* (removing unnecessary data).
- **Data Warehouse Layer:** Includes *denormalization* (to improve query performance) and *aggregation* (to summarize data).

Example: An address may be processed in three steps:

- **Formatting:** Splitting a single string into fields (name, address, city).
- **Standardization:** Converting values into a standard format.
- **Correction:** Fixing errors such as typos.

Phase 4: Loading

The final phase loads the processed data into the data warehouse. There are two main methods:

- **Refresh:** The existing data in the warehouse is completely rewritten and older data is replaced. This method is normally combined with *Static Extraction* when initially populating the database.
- **Update:** Only the changes are added to the warehouse. Existing data is preserved. This technique is combined with *Incremental Extraction* to keep the data warehouse regularly updated over time.

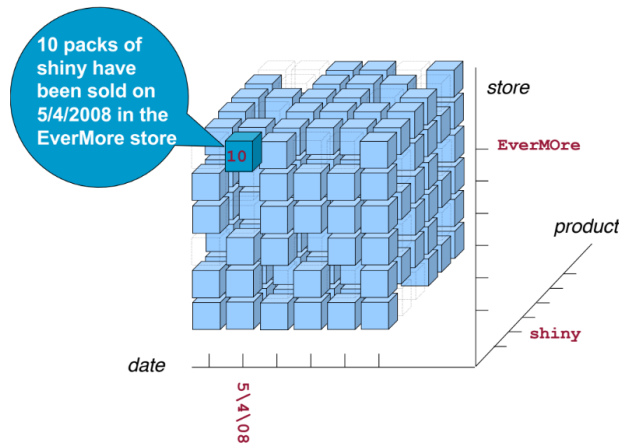
1.5 The Multidimensional Data Model and OLAP

The multidimensional data model provides the principled foundation for how users see the data warehouse. It defines how the virtual representation of the data warehouse is structured and queried.

In this model, the organization-specific ***Facts*** that are the subject of the analysis, are represented as **cubes**. A data cube consists of the following elements:

- **Measures:** Each cell inside the cube contains numerical measures. These numbers quantify the facts from one point of view (e.g., total sales, revenue, or quantity).
- **Dimensions:** Each axis of the cube represents a dimension of interest for the analysis (e.g., time, geography, or product).
- **Hierarchies:** Each dimension can be associated with a hierarchy of dimensional attributes. These hierarchies are used to aggregate (group and summarize) data stored in the cubes at various levels of detail.

Example: A Three-Dimensional Sales Cube To visualize this, imagine a three-dimensional cube used to model supermarket sales. The three axes (dimensions) are **Date**, **Product**, and **Store**. If we locate a specific cell within this cube where the Date is “5/4/2008”, the Product is “Shiny”, and the Store is “EverMore”, the number inside that cell (the measure) might be 10. This simply means: *10 packs of the product ‘Shiny’ were sold on 5/4/2008 in the EverMore store.*

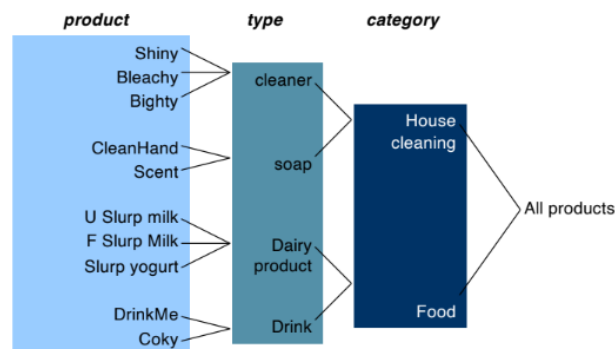


Hierarchies and Functional Dependencies

Dimensions are rarely flat; they contain logical hierarchies driven by strict **functional dependencies**. A functional dependency means that a lower-level attribute belongs to exactly one higher-level attribute.

For example:

- **Product Hierarchy:** $product \rightarrow type \rightarrow category$.
(e.g., Every specific product, like “Shiny”, has exactly one type, like “cleaner”. Every type belongs to exactly one category, like “House cleaning”.)



- **Store Hierarchy:** $store \rightarrow city \rightarrow region$.
(e.g., Every store is located in exactly one city, and every city is located in exactly one region.)

Another common example is the **Virtual-Mall (V-Mall)**. In such a system, you might see these attribute hierarchies:

- **Time:** Time → Month → Quarter → Year
- **Product:** Product → Subcategory → Category
- **Customer:** Customer → Customer City → Customer Region

OLAP (Online Analytical Processing)

OLAP is the most popular way to exploit the vast amounts of information stored in a data warehouse. It provides end users, who often cannot define their exact analysis needs beforehand, the opportunity to explore data interactively using the multidimensional model. Because it requires users to navigate complex business data:

- Users must have an in-depth knowledge of the business data.
- The system must allow for highly complex queries.
- The software interface must be flexible, easy to use, and effective, guaranteeing that users who are not IT experts can still perform advanced analyses.

The OLAP Session

An OLAP analysis is not a single query, but an active **session**. A session consists of a *navigation path* where a user analyzes facts from different viewpoints and varying levels of detail.

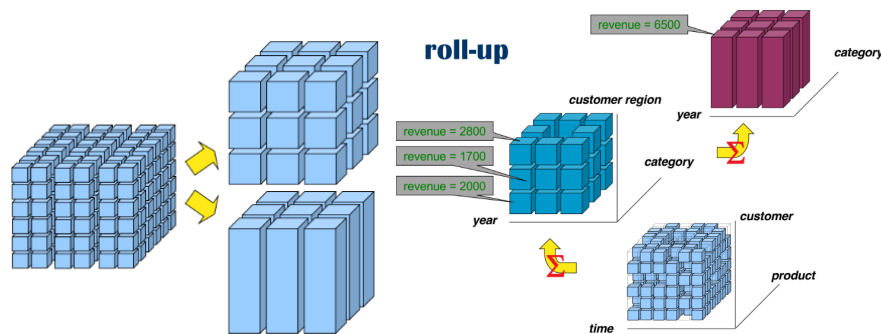
The user's path is translated into a sequence of database queries. Usually, the user does not write these queries from scratch; instead, each new query is defined *differentially* based on the results of the previous step. At every step, an **OLAP operator** is applied to transform the previous query into the new one. The results are always multidimensional and are typically displayed in rich tables featuring multiple headers, colors, and layouts to highlight the data dimensions clearly.

Core OLAP Operators

During an OLAP session, users manipulate the data cubes using several primary operators. Each operator changes the view or the aggregation level of the data to support different analytical needs.

1. **Roll-up:** This operator causes an **increase in data aggregation** by removing a level of detail from an attribute hierarchy. Visually, it compresses the cube. Every macro fact measure created by a roll-up is mathematically the sum (or another aggregate function) of its component fact measures.

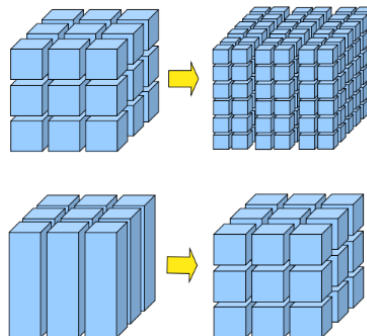
Example: A retail manager is looking at a cube containing daily sales data. They apply a roll-up operator to view the data by "Region" instead of "City". If city A



made \$2000 and city B made \$1700, rolling up to the Region level will yield a combined revenue of \$3700.

2. **Drill-down:** This is the exact inverse of roll-up. It is used to **reduce the level of aggregation** and add detail to the data. Visually, it expands a summarized block into smaller, detailed blocks by stepping down an attribute hierarchy.

Example: A regional director notices that total sales for "Quarter 3" were exceptionally low. They use the drill-down operator on the "Time" dimension to break Quarter 3 down into "July," "August," and "September" to pinpoint exactly which month caused the drop in revenue.



3. Slice-and-Dice:

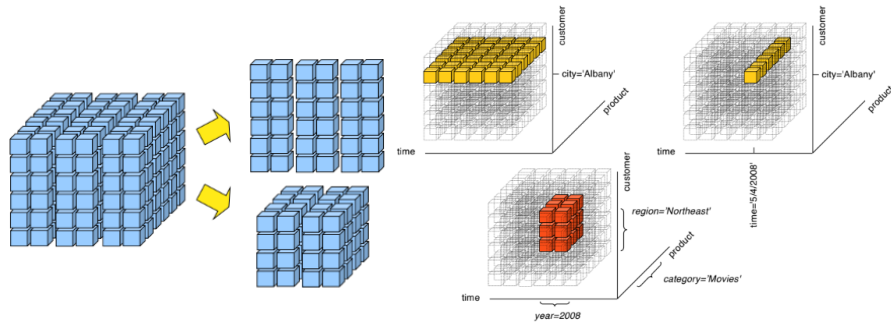
- **Slice:** When you slice data, you decrease the cube's dimensionality by setting one or more dimensions to a single, specific value. This essentially turns a 3D cube into a 2D table (a "slice" of the cube).

Example: An analyst has a 3D cube showing Sales by Product, Region, and Time. By applying a slice where *Time* = '2023', they isolate a flat, 2D spreadsheet showing only the Products and Regions for that specific year.

- **Dice:** Dicing is a generalization of slicing. Instead of picking just one value to drop a dimension, it applies specific constraints or filters on the dimensional attributes (often acting on the hierarchy) to select a smaller "sub-cube." Notice that dicing simply isolates data; it does not aggregate it.

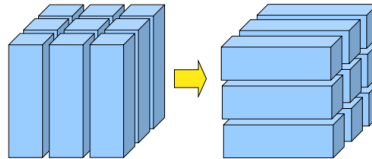
Example: A marketing team wants to evaluate a highly specific market segment. They dice the main cube by setting constraints: *Time* = '5/4/2008', *Region* =

'Northeast', and *Category* = 'Movies'. The result is a smaller 3D cube containing only the data that matches all three criteria.



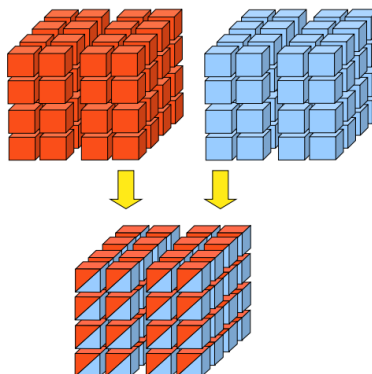
4. **Pivoting:** This operator is used to change the physical structure or orientation of the cube. It provides a different visual perspective of the same data by rotating the axes.

Example: A report currently displays "Regions" as rows and "Months" as columns. An analyst pivots the data to swap them, making "Months" the rows and "Regions" the columns. The underlying data hasn't changed, but the new layout makes it easier to compare the monthly trends side-by-side.



5. **Drill-across:** This operator combines data from two or more different cubes to obtain a new, integrated cube. This is typically done by linking cubes that share one or more common dimensions.

Example: A company has one data cube tracking "Sales" and a completely separate cube tracking "Warehouse Inventory." Both cubes share the "Product" and "Time" dimensions. An analyst performs a drill-across to merge these cubes, allowing them to directly compare the number of laptops sold this month against the number of laptops currently sitting in the warehouse.



1.6 Accessing the Data Warehouse

Once a data warehouse has been built and populated, users need effective tools to access and analyze the data. Different approaches are available, depending on the user's skills and business needs: OLAP, reports, dashboards and data mining.

OLAP (Online Analytical Processing)

As discussed before, OLAP is an interactive approach for data analysis. It allows users to explore data in a multidimensional way.

However, OLAP is mainly designed for analysts, because it requires:

- the ability to think in multidimensional terms,
- knowledge of specific analytical and graphical tools.

Reports

Reports provide structured information in a static and easy-to-read format.

- **No IT Knowledge Required:** Reports are simple to understand and do not require technical skills.
- **Predictable Routine:** They are usually generated at regular intervals (for example, monthly reports).
- **Format:** Reports use standard formats such as tables, line charts (to show trends), and pie charts (to show proportions).

Dashboards

Dashboards are Graphical User Interfaces (GUIs) that display a small set of important data in a clear and concise way.

- **Real-Time Overview:** They provide a quick view of current trends and business performance.
- **Target Audience:** They are mainly used by managers and executives who need a fast overview of the situation.
- **Underlying Mechanics:** Dashboards are based on key *performance indicators* (KPIs), which summarize important aspects of the business.
- **Limitation:** Dashboards are useful for monitoring, but not for deep analysis. For complex investigations, they must be combined with tools such as OLAP.

Data Mining

Data mining is an advanced technique used to discover **hidden information** and **patterns** in large datasets.

When the amount of data is very large, it is difficult for humans to manually identify all relevant patterns. Data mining uses methods from **Artificial Intelligence** and **pattern recognition** to solve this problem.

Some common techniques include: Genetic algorithms, Fuzzy logic, Expert systems, Neural networks.

Unlike traditional queries, where the user asks a specific question, data mining is more automatic. The user only needs to specify:

- *what* type of result to find,
- *where* to look, i.e. which datasets to analyze.

The system then discovers patterns and relationships automatically.

Examples of Business Applications

Data mining is widely used in many fields. Some common applications are:

- **Market Research:** Studying customer behavior and preferences.
- **Marketing Effectiveness:** Analyzing user behavior, for example through click-stream data.
- **Market Segmentation:** Dividing customers into groups based on similar characteristics.
- **Investment Modeling:** Predicting market trends using historical data.
- **Fraud Detection:** Identifying unusual patterns to detect fraud, especially in banking and insurance.
- **Risk Assessment:** Assigning risk levels to customers in financial services.
- **Event Sequence Analysis:** Recognizing patterns in sequences of events, such as detecting system failures.
- **Healthcare and Science:** Supporting clinical analysis and studying epidemiological data.

1.7 Implementing the Multidimensional Data Model

To implement a multidimensional data model, organizations can choose between different approaches. The most common ones are **ROLAP**, **MOLAP**, and their combination, **HOLAP**.

1.7.1 ROLAP (Relational OLAP)

ROLAP stands for **Relational OLAP**. It implements the multidimensional model using traditional Relational Database Management Systems (RDBMS).

ROLAP is widely used because relational databases are well-known, flexible, and able to manage very large amounts of data and users.

However, relational systems do not naturally support concepts such as dimensions, measures, or hierarchies. This creates some challenges:

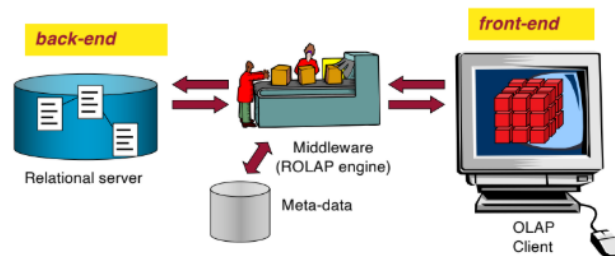
- **Schema Design:** Special schemas, such as the **Star Schema**, must be designed to force the relational database to act like a multidimensional one.
- **Performance Issues:** Analytical queries require many *join* operations on large tables, which are expensive. To improve performance, data is often **denormalized**.

Even with optimizations like *materialized views*, it means that some queries are created and stored in advance, ROLAP is usually slower than systems designed specifically for multidimensional analysis.

Architecture

ROLAP uses a middleware component called the **ROLAP Engine**, between relational back-end servers and front-end components.

When a user submits a request for multidimensional analysis, the ROLAP engine translates multidimensional queries into SQL, sends them to the relational database, and then converts the results from SQL back into a multidimensional format.



1.7.2 MOLAP (Multidimensional OLAP)

MOLAP uses a native multidimensional model, where multidimensional data and operations are directly represented and stored.

Events and Data Cubes

In MOLAP, data is organized using events:

- **Primary Event:** A detailed real-world occurrence of fact, defined by a unique combination of dimension values and a measure, i.e. identified by one n-tuple made up of a value for each dimension. For example, *10 packages of Shiny detergent were sold for total sales of \$25 on 10/10/2008 in the SmartMart store.*
- **Secondary Event:** An aggregation of multiple primary events based on selected dimension values, used for higher-level analysis.

The process can be described as follows:

1. Select specific values for some dimensions (for example, *city = Rome, month = May*).
2. Identify all primary events that match these selected values.
3. Group these events together and apply an aggregation function (such as sum or average).

Therefore:

- Each combination of dimension values (an *n-tuple*) defines a group of primary events.
- This group corresponds to a **secondary event**.
- The value of each measure (for example, total sales) is obtained by combining the values of the same measure from all the corresponding primary events.

Hierarchies define how primary events can be aggregated and selected for analysis. Each hierarchy organizes a dimension into multiple levels of detail.

For example, a time hierarchy may be:

$$day \rightarrow month \rightarrow year$$

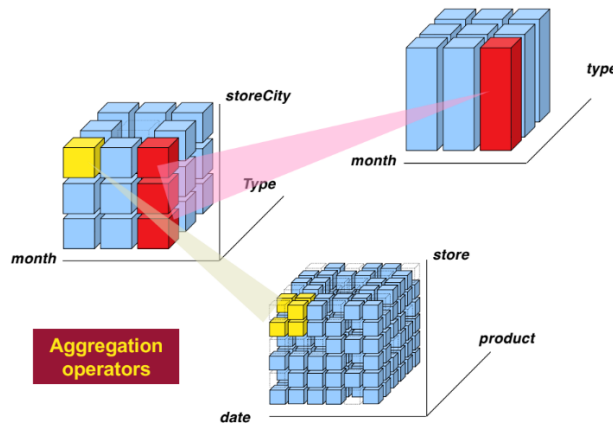
The level where the hierarchy starts (e.g., *day*) represents the finest level of detail, i.e. the highest granularity. As we move up in the hierarchy, data becomes more aggregated and the level of detail decreases, meaning that the granularity becomes coarser.

Therefore, hierarchies are essential for supporting decision-making, because they allow users to analyze data at different levels of detail.

Data is stored in a structure called a **Data Cube**. It contains:

- a primary cube including all the primary events, i.e. the facts,
- several secondary, coarse-grained cubes with aggregated data.

All the secondary cubes do not need to be physically available because you can calculate them by aggregating their primary cube along one dimension. Nevertheless, you may need to store some or all the secondary cubes to improve the query response time.



Example: MOLAP Aggregation (Roll-up) The **roll-up** operation aggregates data to higher levels:

- **The Base Cube (Highest Detail):** The initial multidimensional space is defined by three granular dimensions, it means we start from detailed data (e.g., date, store, product). Each individual cell inside this cube represents a single *primary event* (e.g., selling a specific brand of detergent at a specific store on a specific day).
- **First Aggregation (Time and Geography):** By applying the roll-up operation, data is aggregated along two hierarchies simultaneously: *date* rolls up to *month*, and *store* rolls up to *storeCity*. Consequently, multiple detailed daily sales events are summed together to form a single, aggregated block representing the total monthly sales for that entire city.
- **Second Aggregation (Product):** The data can be further rolled up along the *product* dimension into a broader *type* (e.g., grouping specific brands of soap into a general “Cleaners” category). The specific product sales are summed into one large block representing the entire product category.

These aggregated results form **secondary cubes**, which allow fast answers to complex queries.

Storage and Performance

MOLAP stores data cubes using **arrays**, where each position represents a combination of dimension values. This allows very fast data access, because it avoids expensive join operations.

Advantages: Very high query performance.

Disadvantages:

- No universal standard for the model
- No standard query language like SQL (MDX is commonly used)
- Slow data loading process
- Limited scalability for many dimensions or very large domains

The Problem of Sparsity and HOLAP

A key problem in MOLAP is **sparsity**. Many possible combinations of dimension values do not correspond to real events, but space is still reserved for them. Indeed, in a multidimensional model a set of coordinates exists for every *possible* event, even if that event never actually occurred (e.g., a specific product that was not sold in a specific store on a specific day).

In practice:

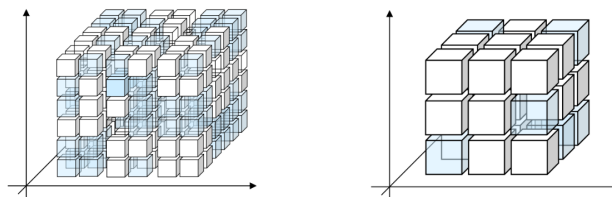
- **ROLAP:** Stores only existing data, so it naturally avoids this problem.
- **MOLAP:** Must manage many empty cells, which requires additional techniques to reduce space usage.

Hybrid OLAP (HOLAP) Solutions

To solve the sparsity problem, **HOLAP** combines ROLAP and MOLAP approaches. We can group cubes into “chunks”, and classify these as *sparse* (mostly empty) or *dense* chunks (lots of data):

Common strategies include:

1. **Chunk-based:** Dense data is stored in MOLAP arrays, while sparse data is stored in ROLAP tables to save space.
2. **Aggregation-based:** Detailed data is stored in ROLAP (primary cubes), while aggregated data is stored in MOLAP (secondary cubes).
3. **Frequency-based:** Frequently used data is stored in MOLAP, while rarely used data remains in ROLAP.



1.8 Design Methodology

Building a data warehouse is a complex task. Many organizations do not have enough experience or technical skills to manage such projects correctly. For this reason, the lack of a clear methodology is one of the main causes of failure in data warehousing projects.

A design methodology provides a structured guide for development. It is based on previous experiences and on the analysis of past errors. Using a proven methodology helps reduce the risk of failure.

1.8.1 Facts

In data warehousing, **facts** are the central elements used for decision-making. A fact represents a type of measurable event inside the organization, such as a sale, a hospital admission, or a bank transaction. Facts must be identified early, during the *Requirement Analysis* phase.

When defining a fact, designers must consider:

- **Fact Dimensions:** The perspectives used to analyze the fact (for example, analyzing a sale by time, store, and product).
- **Fact Granularity:** The level of detail stored in the data mart. This requires a balance between:
 1. *Maximum Flexibility:* Keeping very detailed data, as in operational systems, allows more precise queries.
 2. *High Performance:* Using aggregated data improves query speed but reduces detail.
- **Historical Interval:** The time period covered by the fact (for example, storing data for the last five years).

Facts and data marts depend on the application domain. Different industries use different types of facts.

Application Field	Data Mart	Facts
Business, manufacturing	Supplies	Purchases, stock inventory, distribution
	Production	Packaging, inventory, delivery, manufacturing
	Demand management	Sales, invoices, orders, shipments, complaints
	Marketing	Promotions, customer retention, advertising campaigns
Finance	Banks	Checking accounts, bank transfers, mortgage loans, loans
	Investments	Securities, stock exchange transactions
	Services	Credit cards, bill payment through standing orders
Health service	Division	Admissions, discharges, transfers, surgical operations, diagnosis, prescriptions
	Accident & emergency	Admissions, tests, discharges
	Epidemiology	Diseases, outbreaks, treatments, vaccinations
Transportation	Goods	Demand, supply, transport
	Passengers	Demand, supply, transport
	Maintenance	Operations
Telecommunications	Traffic management	Network traffic, calls
	Customer relationship management	Customer retention, complaints, services

Figure 1.1: Typical Facts Across Different Industries

Conceptual Modeling for Data Warehouses

In software engineering, a good conceptual design is essential to build a database that satisfies business requirements.

However, data warehousing introduces a specific problem:

- The multidimensional model is widely used at the logical level, but there is no standard methodology for conceptual modeling.
- In relational databases, the **Entity/Relationship (ER) model** is the standard tool for design.
- However, the ER model is not well suited for data warehouses, because it focuses on reducing redundancy and does not naturally support hierarchies and aggregations.

As a result, designers must adapt traditional models or use specific techniques designed for data warehousing.

1.9 The Dimensional Fact Model (DFM)

Since the ER model is not suitable for multidimensional design, specialized models are required.

The **Dimensional Fact Model (DFM)** is a graphical conceptual model designed specifically for data marts.

In DFM, the schema is represented as a set of **fact schemata**. Each fact schema describes: facts, measures, dimensions, hierarchies.

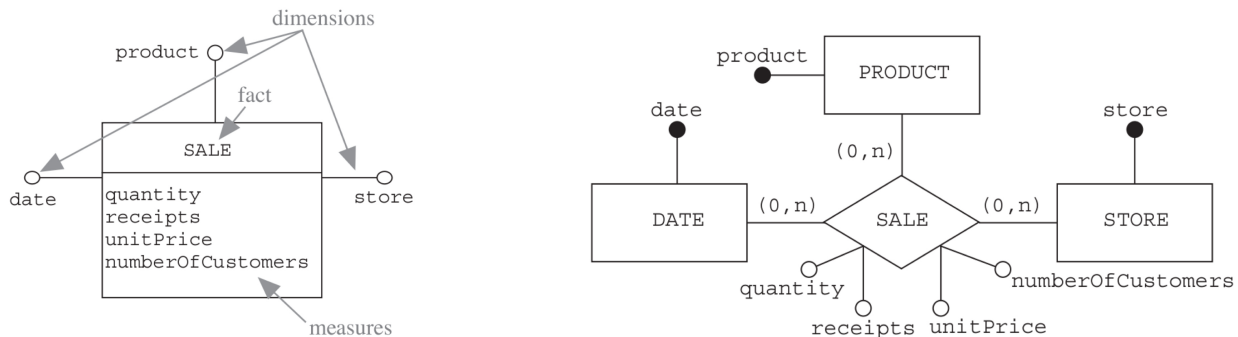
Basic Constructs of the DFM

1. A **fact** is a concept relevant to decision-making processes. It typically models a set of events taking place within a company (e.g., sales, shipments, purchases, or complaints). It is that a fact have dynamic properties or evolve in some way over time.

Note: A fact can sometimes have no numerical measures. This is known as a “factless fact,” and it is used when the business is only interested in recording the occurrence of an event.

2. A **measure** is a numerical property of a fact and describes a quantitative fact aspect that is relevant to the analysis. For example, if the fact is a “Sale,” the measures might include the *number of units sold*, the *unit price*, and the *total receipts*.
3. A **dimension** is a fact property with a finite domain and describes an analysis coordinate of the fact. For a “Sale” fact, the typical dimensions are the *product* sold, the *store* where it was sold, and the *date* of the transaction.

Note: A fact expresses a **many-to-many association** between its dimensions.

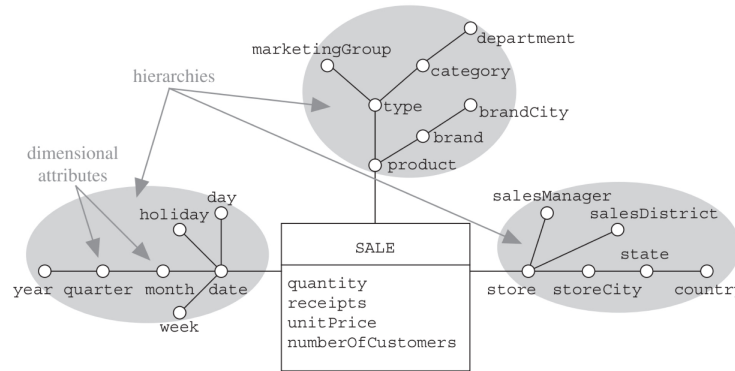


Dimensional Attributes and Hierarchies

Dimensions rarely exist in isolation, they are usually described by broader categories.

- The general term **dimensional attributes** stands for the dimensions and other possible attributes. It means that includes the root dimensions and any other possible attributes with discrete values, that describe them. For instance, a *product* dimension is further described by its *type*, the *category* it belongs to, its *brand*, and the *department* in which it is sold.
- A **hierarchy** is a directed tree whose nodes are dimensional attributes and whose arcs model many-to-one associations between dimensional attributes. The *dimension* is positioned at the root of the tree. For example, many individual products map to one brand, and many brands map to one category.

Visually, in a DFM diagram, the fact is drawn as a central box containing its measures. The root dimensions radiate outward from this box as circles, which then branch out further into hierarchical trees of dimensional attributes.



DFM vs. ER Diagrams

It is possible to use traditional Entity/Relationship (ER) diagrams to model a data warehouse, but this approach is not recommended. In an ER diagram, a fact is usually represented as a many-to-many relationship (diamond) connecting multiple entities. When hierarchies are added, the diagram becomes very complex and difficult to read. The main problem is that, even if the ER model can represent facts, dimensions, and measures, it does not clearly distinguish them. In particular, dimensions look like normal entities, so it is hard to understand the analytical structure.

The **Dimensional Fact Model (DFM)** solves this problem by using a clear and specific graphical representation designed for multidimensional analysis.

1.9.1 Naming Conventions

When designing a fact schema, it is important to define clear and consistent naming conventions.

The first rule is that each dimensional attribute inside a fact schema must have a unique name.

If two attributes have similar names, it is necessary to distinguish them using a prefix. This prefix should refer to the name of the dimensional attribute that comes before them in hierarchies. For example, instead of using the generic name *City*, it is better to use names like *brandCity* or *storeCity*.

Another important rule is that attribute names should not refer to the specific fact. For this reason, names like *shippedProduct* or *shipmentDate* should be avoided, and more general names should be preferred.

Finally, if an attribute has the same meaning in different fact schemata, it must always use the same name. This ensures consistency across the entire data warehouse.

1.9.2 Descriptive and Cross-Dimensional Attributes

A **descriptive attribute** is determined by a dimensional attribute of a hierarchy and specifies a property of this dimensional attribute.

Descriptive attributes often are tied to dimensional attributes by one-to-one associations and do not actually add useful levels of aggregation. They can anche be directly connected to a fact if they describe primary events, though they are never used to identify those events or perform calculations.

Note: In an ER representation of such model, you can model descriptive attributes as attributes of the entities corresponding to dimensional attributes or the relationship modeling the fact.

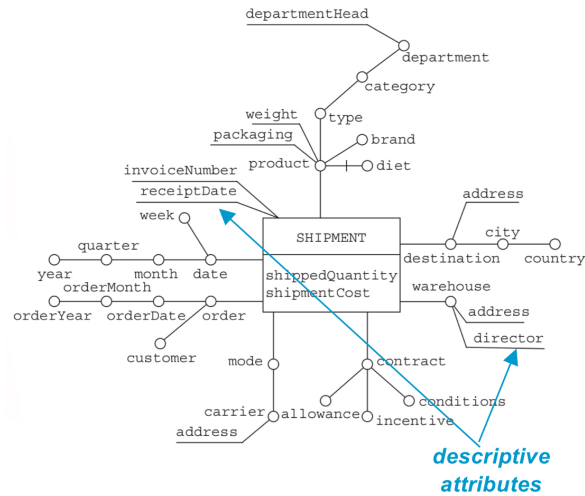


Figure 1.2: Descriptive attribute

A **cross-dimensional attribute** is a descriptive attribute whose value is defined by the combination of two or more dimensional attributes. For example, a product Value Added Tax (VAT) depends both on the product category and on the country where the product is sold.

Note: In an ER representation of such model, a cross-dimensional attribute is modeled as an attribute of a relationship between the participating entities (e.g., CATEGORY and COUNTRY), where both entities participate with a cardinality of $(1, n)$.

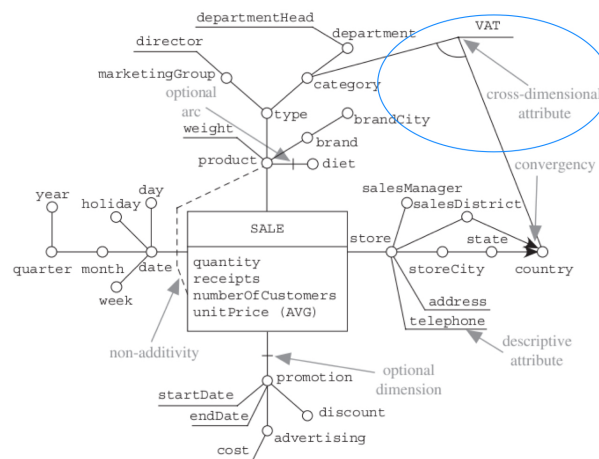


Figure 1.3: Cross-dimensional attribute

1.9.3 Optional Dimensions and Attributes

Real-world data is often incomplete, which is why DFM allows certain arcs to be marked as *optional*. This optionality can be applied to dimensional attributes, measures, or even entire

dimensions.

- **Optional Attributes:** If the optionality is applied to a *dimensional attribute* (visually indicated by a small dash across the connecting line), it means that the attribute, along with all of its possible descendants, might be undefined for certain instances of the fact.
- **Optional Measures:** When a *measure* itself is optional, meaning it might not be recorded for every single event, its name is annotated with an asterisk (for example, `shipmentCost*`).

Note: In an ER representation of such model, the minimum cardinality of the association from PRODUCT to DIET is 0 rather than 1.

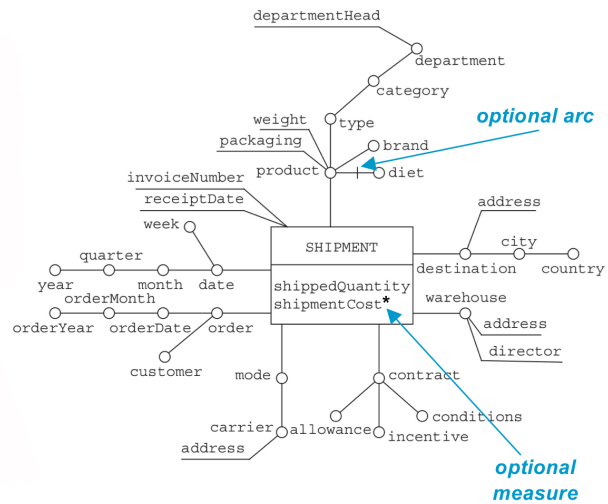


Figure 1.4: Optional attribute

- **Optional Dimensions:** When a dimension d is optional, it implies that some primary events exist that are perfectly identified only by the other remaining dimensions.

Note: In an ER representation of such model, the SALE has to be reified and identified by its participation to the relationships towards DATE, PRODUCT, and STORE. Also, it has a (0,n) participation to a relationship towards PROMOTION.

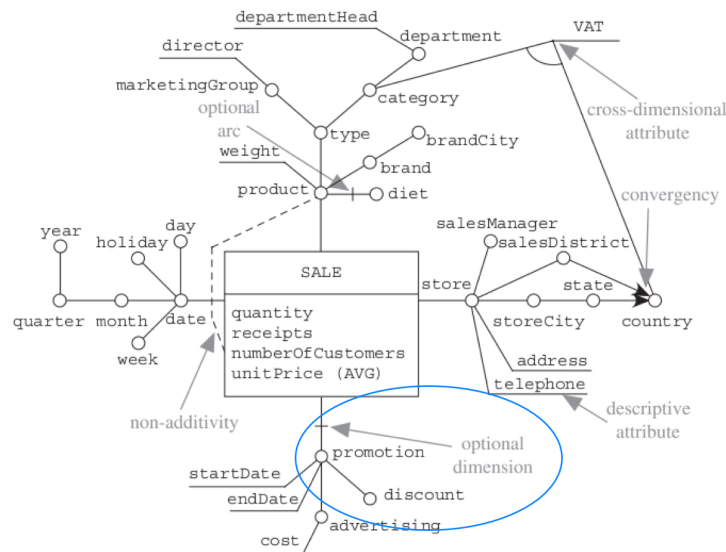


Figure 1.5: Optional dimension

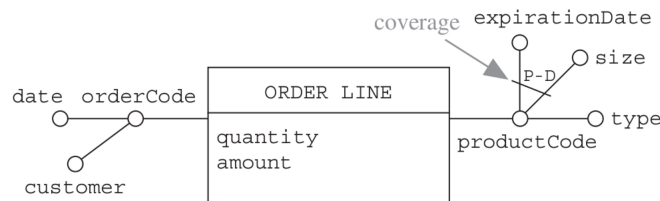
1.9.4 Coverage and Convergence

When building a Dimensional Fact Model (DFM), you often encounter situations where hierarchies are not strictly linear. To handle branching and intersecting paths, the DFM relies on two key concepts: coverage and convergence.

A **coverage** can be specified on two or more arcs that exit from the same dimensional attribute *A*.

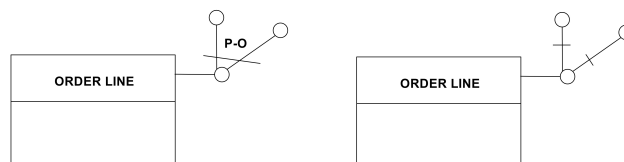
Coverage rules define how the values of attribute *A* map to the values of its child attributes. These rules are built upon two main pairs of conditions:

- **Total (T) vs. Partial (P):** A *total* coverage dictates that every single value of *A* must be associated with at least one child attribute. Conversely, a *partial* coverage allows for situations where some values of *A* have absolutely no defined children.
- **Disjoint (D) vs. Overlapping (O):** A *disjoint* rule ensures that any value of *A* is associated with a maximum of one child attribute. An *overlapping* rule is more flexible, allowing a specific value of *A* to link to values in two or more of its child attributes at the same time.



By combining these conditions, we establish four distinct patterns of coverage:

1. **T-D (Total-Disjoint):** The strictest form. Every value of *A* is associated with *exactly one* child attribute.
2. **P-D (Partial-Disjoint):** A value of *A* is associated with *at most one* child attribute (meaning it can have one or zero).
3. **T-O (Total-Overlapping):** A value of *A* is associated with *at least one* child attribute, but could be linked to multiple.
4. **P-O (Partial-Overlapping):** This pattern imposes essentially no constraints; it is pure optionality. In practice, a P-O coverage is logically equivalent to simply having multiple, separate optional dimensional attributes branching off from *A*.



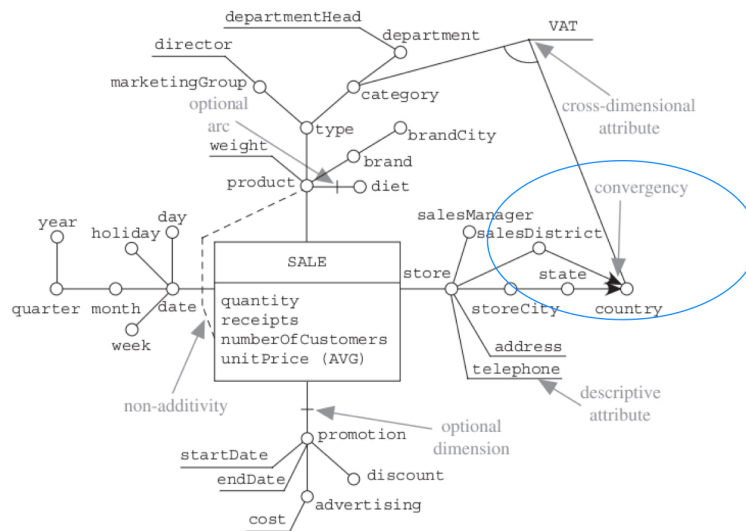
Note: In an ER representation, coverage is expressed through external constraints regulating the participation of the entity corresponding to the attribute A to the relationships towards the entities corresponding to the attributes of the coverage.

While coverage deals with hierarchies branching out, **convergence** deals with paths coming back together.

Convergence is an integrity constraint. It occurs when two or more distinct hierarchical paths eventually lead to the exact same dimensional attribute, and logically, the results of both paths must agree.

A classic example involves geography and sales regions. If you determine a store's country by tracing its path through *storeCity* and then *state*, that country must be identically the same as the country determined by tracing the store's *sales District*. The paths diverge, but they must converge on the same absolute value.

ER Modeling Note: In an ER representation of such model, convergence has to be specified through an external constraint.



While convergence is an essential tool for modeling complex, parallel hierarchies, it is important to recognize and eliminate *redundant* convergences. A convergence is considered redundant when one hierarchical path connects two dimensional attributes directly, while an alternate path connects those exact same attributes through one or more intermediate nodes.



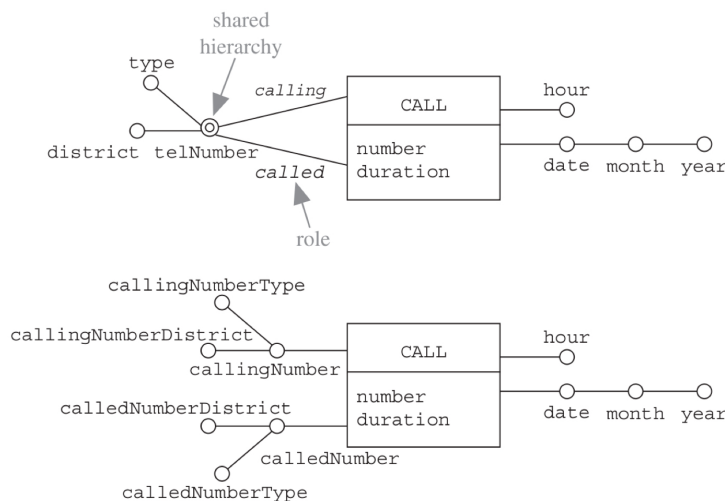
In data modeling, this direct link has no reason to exist because of the transitive property inherent to functional dependencies. For instance, if an attribute A functionally determines

an intermediate attribute B , and B functionally determines attribute C , it is completely obvious mathematically that A also determines C . Therefore, drawing an additional direct arc from A to C provides no new structural information. To maintain a clean, accurate, and optimized DFM representation, the redundant direct path must be removed, leaving only the sequence that properly captures the intermediate levels of the hierarchy.

Streamlining the Schema: Shared Hierarchies

In many business scenarios, different dimensions within the same fact schema follow the exact same structure. For example, a telephone call involves both a “calling” number and a “called” number; both belong to a district and have a specific line type. Similarly, a shipment has an “order date” and a “shipment date,” both rolling up into months and years. To keep the schema clean and easy to read, the Dimensional Fact Model (DFM) uses a **shared hierarchy**. This approach avoids repeating the same hierarchy branches multiple times. Instead, the hierarchy is drawn once, and multiple lines (arcs) connect the fact to the same root attribute.

To avoid confusion, you must define the specific context for each connection. When a hierarchy is shared, a **role** is specified using an arc label. For instance, the arcs connecting a **CALL** fact to the **telNumber** dimension are labeled as *calling* and *called* to distinguish their functions.

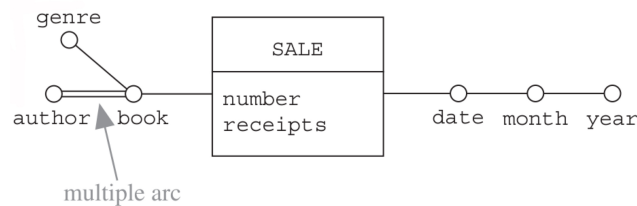


Modeling Many-to-Many Relations: Multiple Arcs

Standard arcs in DFM represent many-to-one relationships (for example, many products belong to one category). However, real-world data often involves many-to-many relationships. To model this, DFM introduces the **multiple arc**, which is visually represented by a double line or an arrow on the arc.

A multiple arc allows one attribute to be linked to several values of a parent attribute. A common example is the relationship between books and authors: while a book has only one

genre, it can have more than one author. In this case, the connection between **book** and **author** is drawn as a multiple arc.

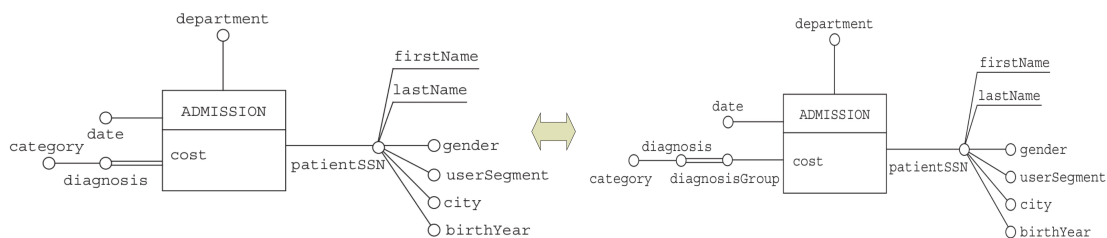


Multiple Arcs Entering a Dimension

A special case occurs when a multiple arc connects a fact directly to a dimension. Normally, a single fact is identified by one value for each dimension.

If a multiple arc enters a dimension—for example, a hospital **ADMISSION** fact linked to several **diagnoses**—the fact is actually identified by a “group” of those elements. Conceptually, the dimension value for that fact becomes a *diagnosis group* rather than a single diagnosis.

Note: When moving from a DFM schema to an Entity-Relationship (ER) model, a multiple arc always represents a many-to-many relationship.



1.9.5 Additivity and Measure Types

In conceptual modeling, aggregation is the process of combining measure values from primary events into summarized secondary events. This requires choosing a suitable operator. Based on how they behave, measures are divided into three categories:

- **Flow Measures:** These refer to a timeframe and are recorded cumulatively at the end of that period. Examples include daily sales or annual births. Since these represent totals, all standard operators (**SUM**, **AVG**, **MIN**, **MAX**) are valid across all hierarchies.
- **Level Measures:** These represent a state at a specific point in time, such as inventory levels or city population. While you can sum these across non-temporal hierarchies (e.g., adding inventory from different stores, add up the inventory in Rome to that in Milan), it is illogical to sum them over time. Therefore, only **AVG**, **MIN**, and **MAX** are valid for temporal hierarchies.

- **Unit Measures:** These are expressed in relative terms, like unit prices or discount percentages. Because they are ratios, the **SUM** operator is never used (e.g., if bread costs €2 and milk costs €1.50, the average unit price is not the sum of the two (€3.50)). They can only be aggregated using **AVG**, **MIN**, and **MAX** across any dimension.

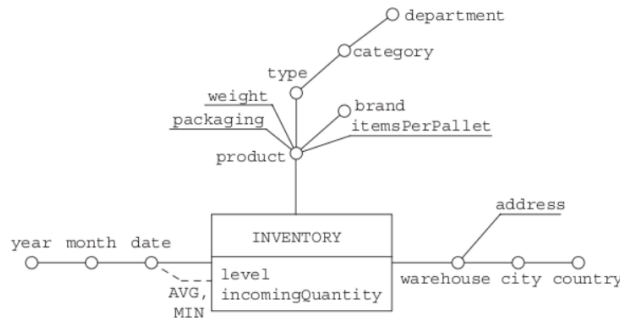
Measure Type	Temporal Hierarchies	Nontemporal Hierarchies
Flow Measures	SUM, AVG, MIN, MAX	SUM, AVG, MIN, MAX
Level Measures	AVG, MIN, MAX	SUM, AVG, MIN, MAX
Unit Measures	AVG, MIN, MAX	AVG, MIN, MAX

Table 1.1: Valid Aggregation Operators for Measures

1.9.6 Additive vs. Non-Additive Dimensions

A measure is **additive** along a dimension when the **SUM** operator can be used to aggregate its values. If summing produces an incorrect result, the measure is **non-additive** for that specific dimension.

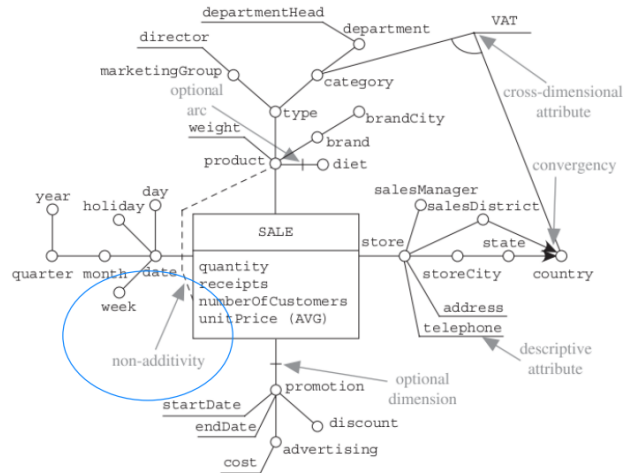
In DFM notation, if a measure is non-additive along a dimension, it is connected by a *dashed line*. This line is labeled with the specific operators that are allowed. For example, an inventory level is non-additive along the **date** dimension; therefore, it uses a dashed line labeled with “AVG, MIN” to prevent incorrect summations over time.



1.9.7 Non-Aggregable Measures and Shorthand Notation

A measure is **non-aggregable** if it cannot be summarized at all along a dimension. For example, in a **SALE** schema, the **numberOfCustomers** (number of receipts issued for a product in a certain date in a store) is non-additive along the **product** dimension. Since one receipt can contain many products, summing the **numberOfCustomers** along the product dimension would mean to count a customer several times. In DFM, this is shown as a dashed line *without labels*.

If a measure follows the same aggregation rule across all dimensions, DFM uses a **shorthand notation**. Instead of multiple dashed lines, the allowed operator is written directly next to the measure name inside the fact box, such as **unitPrice (AVG)**.

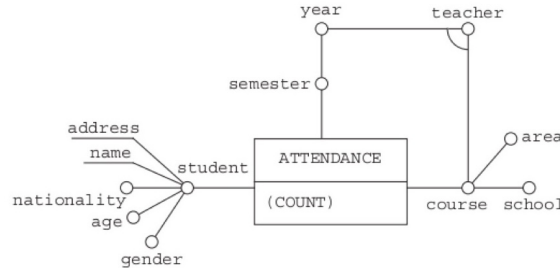


1.9.8 Special Types of Fact Schemata

In addition to standard models, the Dimensional Fact Model (DFM) includes specific structures for events that lack numerical measures or for analysis that spans multiple processes. These are known as *empty* and *overlapping fact schemata*.

Empty Fact Schemata

A fact schema is defined as **empty** if it contains no measures. While most facts track numerical data like price or quantity, some business needs only require recording that an event happened within an application domain.



For example, when tracking student attendance, the event is the presence of a student in a class. There is no numerical “amount” to record. In these cases, the DFM uses a pseudo-measure called **(COUNT)**. This operator shows that a summarized event represents the total number of primary events related to it (such as the total number of times a student attended). Indeed, it indicates that the information represented by a secondary event is the number of primary events corresponding to it.

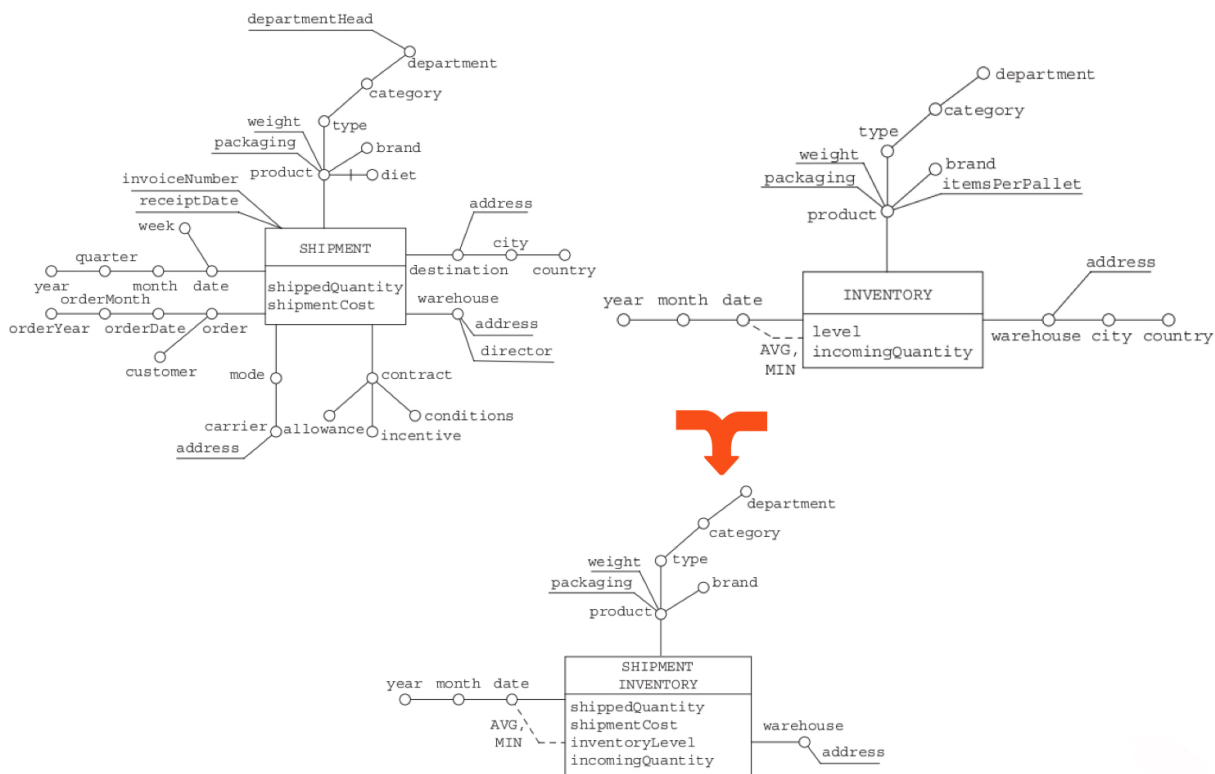
Overlapping Fact Schemata and Drill-Across Queries

Generally, different fact schemata represent different business processes. However, users often need to compare measures from two or more related facts. In OLAP, this is called a **drill-across** query. To make these queries possible, we define an **overlapping schema** using three rules:

1. **Union of Measures:** The new schema includes all measures from the original schemata.

2. **Intersection of Hierarchies:** The schema only keeps the dimensional attributes that appear in all involved hierarchies (i.e., that are common).
3. **Intersection of Domains:** The domain of each shared attribute is the intersection of the domains from the original schemata.

Example of Overlapping: If we overlap a **SHIPMENT** fact (containing date, product, warehouse, destination, and contract) with an **INVENTORY** fact (containing date, product, and warehouse), the resulting schema merges all measures but only keeps the shared dimensions: **date**, **product**, and **warehouse**. Specific dimensions like “contract” are removed to ensure the comparison is valid.



1.9.9 Integrity Constraints

The DFM uses graphical symbols, but these cannot always represent complex business rules. To solve this, we can add **integrity constraints** as separate sentences next to the diagram. Common types of integrity constraints include:

- **Conditions on missing values:** Rules about when dimensional attributes can be null.
- **Functional dependencies:** Defined relationships between attributes not shown by the hierarchy arcs.

- **Fact identification rules:** Stricter rules for identifying facts, such as using only a specific subset of dimensions as a unique identifier.

1.10 Logical Design

After completing conceptual modeling, the next phase is logical design. While conceptual modeling is independent of technology, logical modeling depends on the specific architecture chosen for the data mart.

To represent a multidimensional structure logically, designers use two main models:

- **MOLAP (Multidimensional On-Line Analytical Processing):** This stores data in intrinsically multidimensional structures, like multidimensional vectors.
- **ROLAP (Relational On-Line Analytical Processing):** This uses the standard relational model (tables) to represent multidimensional data.

In data warehousing, logical design often follows the **ROLAP approach**. The goal is to translate the conceptual schema (DFM) into a logical schema using the relational model. This is typically done using two patterns: the **Star schema** or the **Snowflake schema**.

1.10.1 Star Schema

Relational multidimensional modeling is primarily based on the **star schema**. It consists of two main types of tables:

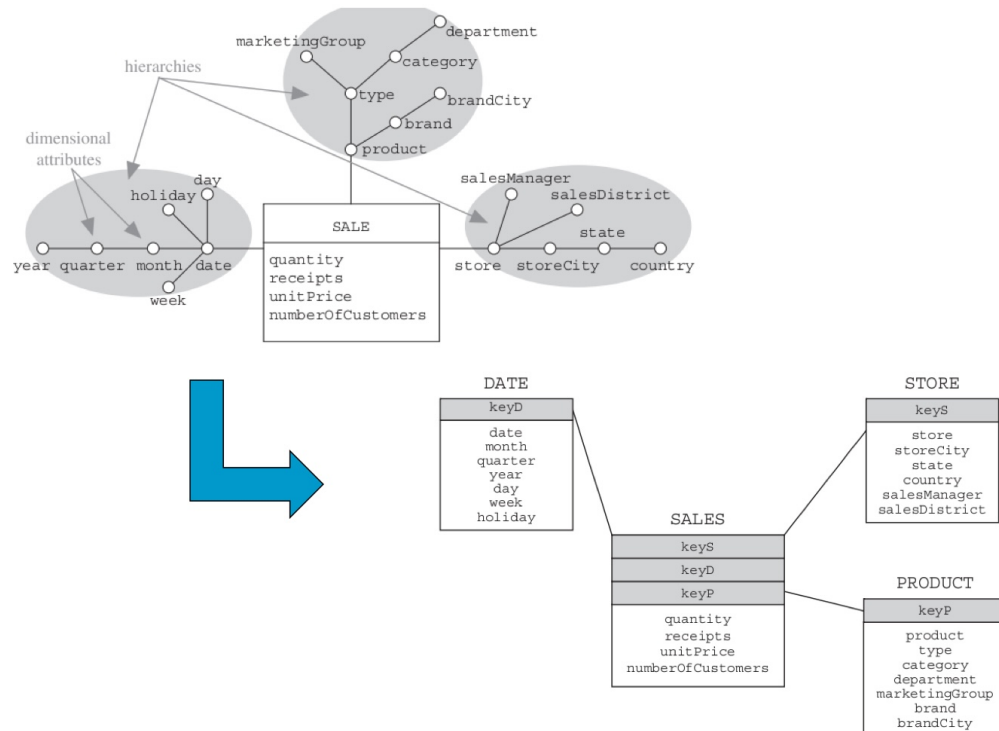
1. A set of relations $DT_1, \dots, DT_n$ called **dimension tables**. Each of them corresponds to a dimension from the conceptual model. It uses a primary key k_i (usually a surrogate key) and includes a set of attributes describing its dimension at different aggregation levels.
2. A **fact table** FT (i.e., the central table) referencing all the dimension tables. Its primary key is a composition of foreign keys $(k_1, \dots, k_n)$ pointing to the dimension tables. It also contains an attribute for every measure.

The conversion from a DFM to a Star Schema is a visual mapping process:

- The central **fact** in the DFM becomes the **Fact Table**, holding all numerical measures.
- The **hierarchies** branching off become the **Dimension Tables**.

A key feature of the star schema is **denormalization**. Instead of separate tables for each hierarchy level, the entire path is collapsed into one wide table. For example, a single “Product” table contains the product, type, category, and department side-by-side.

To link the schema together, each dimension table is assigned a simple, integer-based surrogate primary key (such as **keyD** for DATE, **keyS** for STORE, and **keyP** for PRODUCT). The central Fact Table then includes all of these keys as foreign keys, and its own primary key is usually the composite of these foreign keys.



Star Schema Instances

SALES is the fact table, while STORE, PRODUCT and DATE are the dimension tables.

SALES	<u>keyS</u>	<u>keyD</u>	<u>keyP</u>	quantity	receipts	
	1	1	1	170	85	
	2	1	2	320	160	
	3	2	3	412	412	
						

STORE	<u>keyS</u>	store	storeCity	state	
	1	COOP1	Columbus	Ohio	
	2	COOP2	Austin	Texas	
	3	COOP3	Austin	Texas	
					

PRODUCT	<u>keyP</u>	product	type	category	brand	
	1	Slurp Milk	Dairy product	Food	Slurp	
	2	Fresh Milk	Dairy product	Food	Fresh	
	3	Slurp Yogurt	Dairy product	Food	Slurp	
						

DATE	<u>keyD</u>	date	month	year	
	1	9/2/2008	9/2008	2008	
	2	10/3/2008	10/2008	2008	
	3	10/5/2008	10/2008	2008	
					

Data instances clarify how the star schema works in practice:

- The **Fact Table** stores numerical measures identified by a unique combination of surrogate keys.
- The **Dimension Tables** contain descriptive text for filtering. Because of denormalization, data is repeated (redundancy), such as repeating “Texas” for every store in that state.

While this redundancy violates traditional relational database normalization rules (like the Third Normal Form), it is intentional and desirable in a data warehouse environment. By storing all related dimensional attributes in a single table, the system avoids complex and computationally expensive table joins during query execution.

Evaluating the Star Schema

The star schema involves specific trade-offs:

- **Denormalization:** Functional dependencies (like $product \rightarrow type$ and $type \rightarrow category$) are flattened into a single table.
- **Performance:** A single join is enough to get all data for a dimension, making queries very fast.
- **Redundancy:** While dimension tables have high redundancy, their size is usually negligible compared to the fact table.
- **Scalability:** If facts are very fine-grained, the fact table becomes massive, which can slow down data access.
- **Efficiency:** There are no **sparsity problems** because the schema only stores rows for events that actually occurred.

Integrity Constraints in the Star Schema

Similar to the conceptual phase with the Dimensional Fact Model (DFM), a logical star schema often requires extra rules that table structures cannot show. These **integrity constraints** are usually added as separate sentences next to the graphical schema.

In a star schema, all attributes are assumed to be “**not null**” by default. If an attribute is allowed to have null values, it is marked with an asterisk (*).

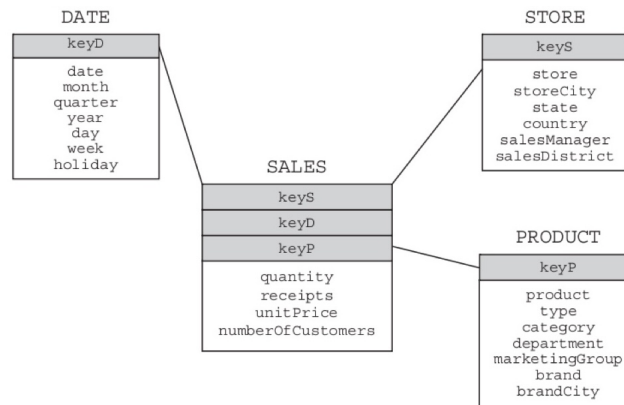
Common types of integrity constraints include:

- **Functional dependencies** between attributes. These are important because dimension tables are denormalized.
- **Stricter conditions for identifying facts.** You can specify that only a specific group of dimensions acts as the identifier for a fact type. In the diagram, these identifying attributes are underlined.

Executing OLAP Queries on a Star Schema

Extracting data in a Relational OLAP (ROLAP) environment is done using standard SQL queries. To get a multidimensional view, you must join the central fact table with its dimension tables.

This join is performed by matching the foreign keys in the fact table to the primary keys of the associated dimension tables.



```

SELECT *
FROM SALES AS FT, PRODUCT AS DT1, STORE AS DT2, DATA AS DT3
WHERE FT.keyP = DT1.keyP AND FT.keyS = DT2.keyS AND
      FT.keyD = DT3.keyD
  
```

Figure 1.6: To link sales data with its dimensions

```

SELECT storeCity, week, type, sum(quantity)
FROM SALES AS FT, PRODUCT AS DT1, STORE AS DT2, DATA AS DT3
WHERE FT.keyP = DT1.keyP AND FT.keyS = DT2.keyS AND
      FT.keyD = DT3.keyD AND DT1.category='Food'
GROUP BY storeCity, week, type
  
```

Figure 1.7: If you need to filter and aggregate data, such as finding the total quantity of “Food” products by city and week, you use the GROUP BY clause.

1.10.2 Snowflake Schema

While the Star schema is optimized for speed through complete denormalization, it ignores standard relational rules. Specifically, its dimension tables are not in the Third Normal Form (3NF) because they contain transitive functional dependencies. For example, in a flat **STORE** table, the store determines the **storeCity**, and the **storeCity** determines the **state**.

To fix the data redundancy this causes, designers can use the **Snowflake schema**. This approach reduces denormalization by eliminating some transitive dependencies, breaking hierarchies into smaller, related tables.

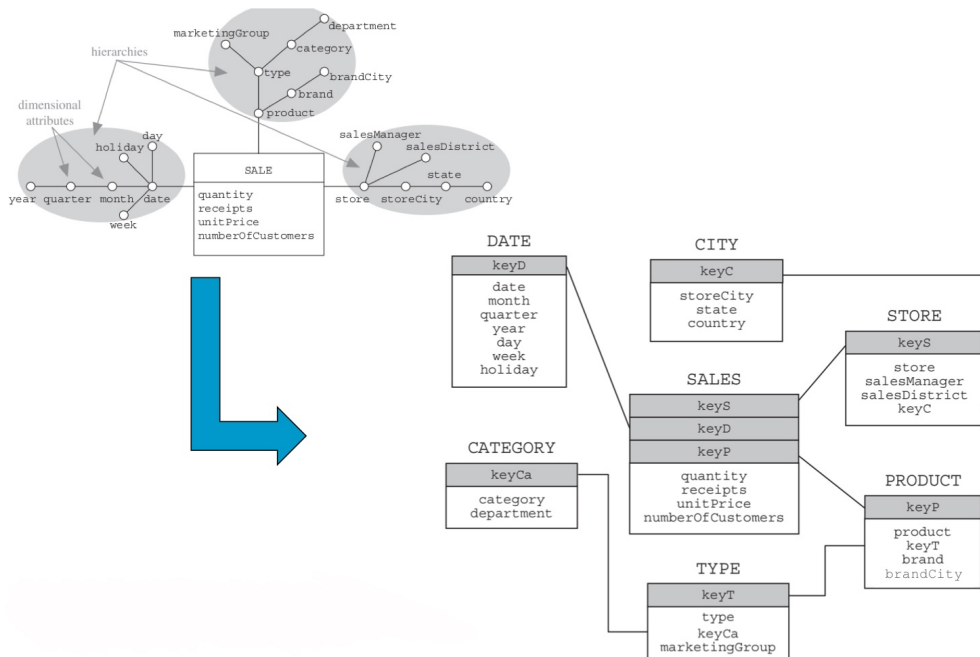
Structure

In a snowflake schema, hierarchies are split into a series of tables that cascade from the central fact table. A dimension table $DT_{i,j}$ in a snowflake schema is characterized by:

- **A primary key $d_{i,j}$:** Usually a surrogate key for unique identification.
- **Dependent attributes:** A subset of $DT_{i,j}$, descriptive attributes, that depend directly on that primary key $d_{i,j}$.
- **Foreign keys:** Keys referencing other dimension tables $DT_{i,k}$ higher in the hierarchy, used to reconstruct the full context.

Because dimensions are split, we distinguish between two types of tables:

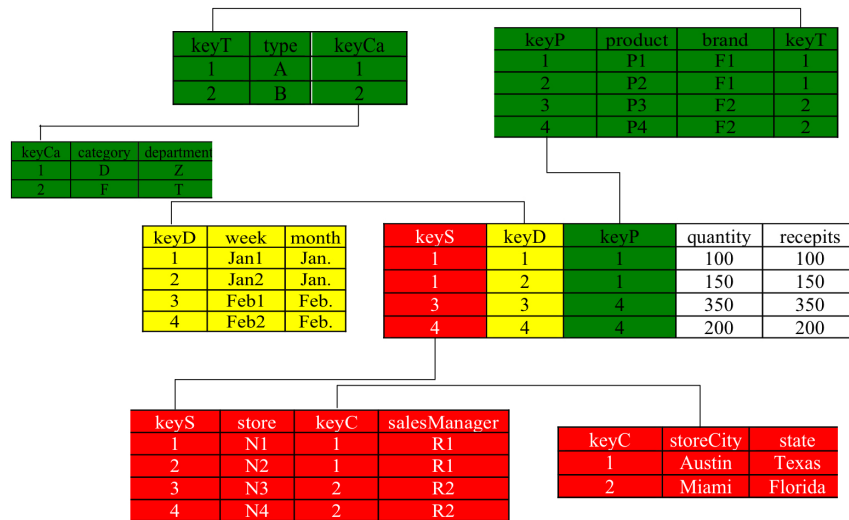
- **Primary Dimension Tables:** Tables whose primary keys are referenced in the fact table (e.g., **PRODUCT** or **STORE**).
- **Secondary Dimension Tables:** Tables referenced by other dimension tables rather than the fact table (e.g., **CITY** or **CATEGORY**).



Snowflake Instances in Practice

Consider the product hierarchy. Instead of one wide table, the snowflake schema might split it into **PRODUCT**, **TYPE**, and **CATEGORY**. The Fact Table points to **PRODUCT**, which points to **TYPE**, which finally points to **CATEGORY**.

Similarly, the **STORE** dimension might link to a secondary **CITY** table. As a result, the city “Austin” and state “Texas” are stored only once in the **CITY** table, instead of being repeated for every store in that city.



Evaluating the Snowflake Schema: Trade-offs

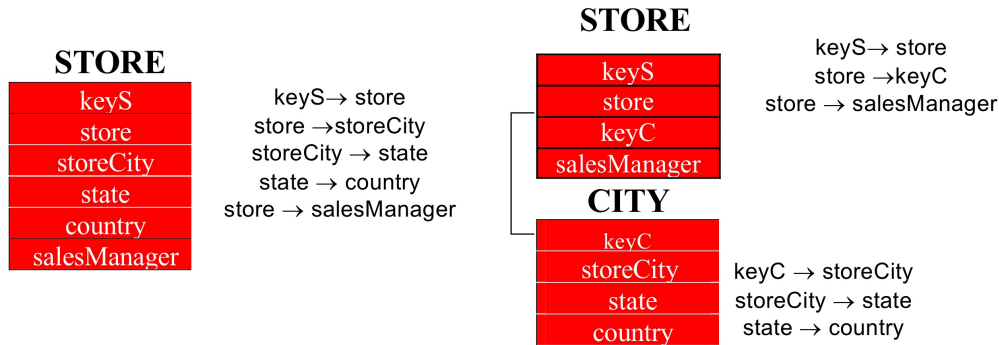
Adopting a snowflake schema involves several practical trade-offs:

- **Storage Efficiency:** Disk space usage decreases because textual descriptions are stored only once rather than being repeated.
- **Key Management:** New surrogate keys are required to link the primary and secondary dimension tables.
- **Query Performance:**
 - Queries using only the fact table and *primary* attributes are streamlined because the tables are smaller.
 - Queries using *secondary* attributes take longer because the database must perform more joins.

The Normalization Process: How to Snowflake

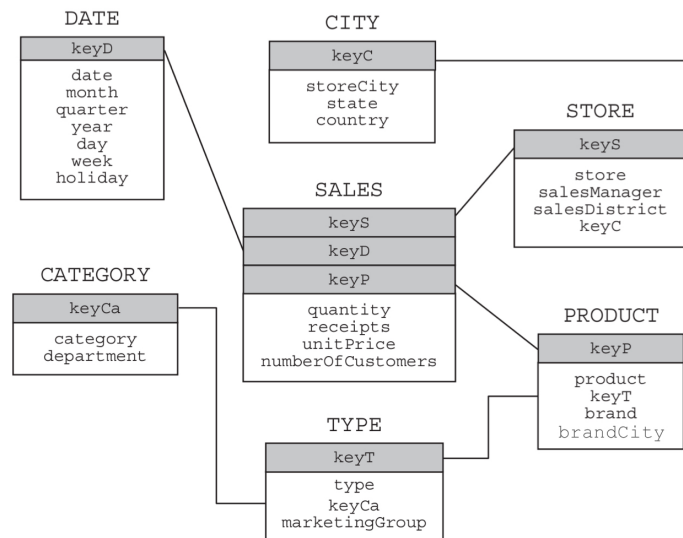
To design a snowflake schema, you must identify which attributes should be moved to a new table. You must move all attributes that directly or transitively depend on the “snowflaking attribute” (the natural key of the new table).

For example, in a **STORE** table, the city determines the state and country ($storeCity \rightarrow state \rightarrow country$). These can be moved to a secondary **CITY** table. The original **STORE** table then keeps only direct dependencies, like the **salesManager**, and a foreign key pointing to the new **CITY** table.



Executing OLAP Queries on Snowflake Schemata

The normalization of a snowflake schema makes Online Analytical Processing (OLAP) queries more complex. Because information is spread across multiple tables, a query must explicitly “stitch” these pieces back together to provide a complete view.



Consider an analytical request to calculate total sales quantity grouped by **storeCity**, **week**, and product **type** for items in the 'Food' category. In a snowflake environment, the SQL statement must be expanded as follows:

```
SELECT storeCity, week, type, sum(quantity)
FROM SALES AS FT, PRODUCT AS DT1, STORE AS DT2, DATA AS DT3,
     TYPE AS DT4, CATEGORY AS DT5, CITY AS DT6
WHERE FT.keyP = DT1.keyP AND FT.keyS = DT2.keyS AND
     FT.keyD = DT3.keyD AND DT6.keyC=DT2.keyC AND
     DT4.keyT=DT1.keyT AND DT5.keyCa=DT4.keyCa AND
     DT5.category='Food'
GROUP BY storeCity, week, type;
```

This query differs significantly from a star schema query in two main ways:

- **Expanded FROM Clause:** The query must include all secondary dimension tables used for filtering or grouping, such as TYPE, CATEGORY, and CITY.
- **Expanded WHERE Clause:** A chain of join conditions is required to traverse the hierarchies. For example, to filter by category, the SALES table joins to PRODUCT, which joins to TYPE (DT4.keyT = DT1.keyT), which finally joins to CATEGORY (DT5.keyCa = DT4.keyCa).

1.11 Performance Optimization via Views

Data warehouses contain vast amounts of information, which can make user analysis difficult. To simplify this, users often apply selection and aggregation to reduce the volume of data they must examine. However, calculating these aggregations on millions of rows is very slow.

To significantly increase performance, designers **calculate the most common aggregate data in advance**. In dimensional modeling, these fact tables containing pre-calculated data are called **views**.

View Classification by Aggregation Level

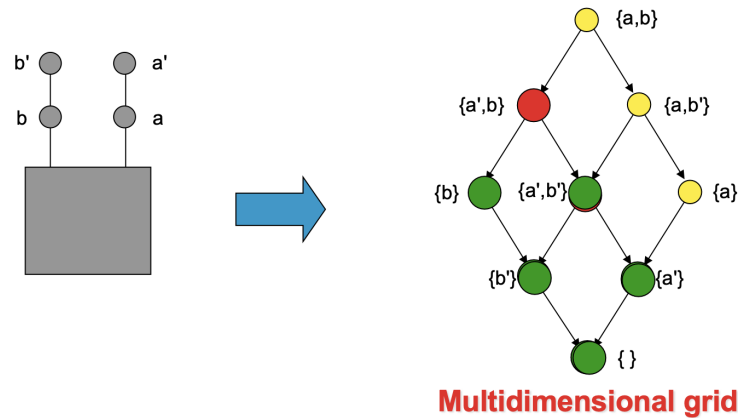
Views are classified based on their level of aggregation, known as a **pattern**. Organizing these patterns creates a clear hierarchy:

- **Primary views:** These correspond to the finest level of detail. They represent the primary, non-aggregated group-by set defined by the fact schema dimensions (e.g., grouping by *product*, *date*, and *store*).
- **Secondary views:** These correspond to any secondary, aggregated pattern where data has been rolled up along the hierarchies (e.g., grouping by *quarter* and *state*).

Using Views to Answer Queries

A pre-calculated view v built over a pattern p is not only useful for queries asking for p , but also for any query with a pattern p' that is **more aggregated** than p (denoted as $p \sqsubseteq p'$). For example, a view summarized by *month* can be used to quickly answer a query about *years*.

The relationships between all possible aggregation levels form a **multidimensional grid**. When a user submits a request, the system uses this grid to find the most efficient pre-calculated view to answer the query.



Aggregation Challenges in Secondary Views

Pre-calculating secondary views can be difficult because you cannot always calculate correct results using only existing aggregated values.

Derived and Support Measures

Sometimes, trying to calculate a value on the fly from an aggregated view leads to errors. For instance, calculating total receipts from aggregated sold quantities and average prices may give an incorrect total. To fix this, you must introduce **Derived Measures**, which are calculated at the primary level and explicitly stored in the secondary view.

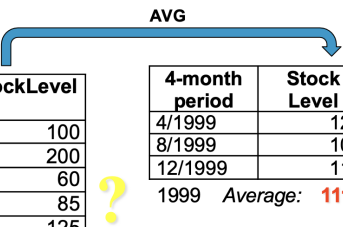
type	product	quantity	unitPrice	receipts
T1	P1	5	1,00	5,00
T1	P2	7	1,50	10,50
T2	P3	9	0,80	7,20
				22,70

		Sum	AVG	
		⇓	⇓	?
type	quantity	unitPrice	receipts	
T1	12	1,25	15,00	
T2	9	0,80	7,20	
				22,20

The choice of mathematical operator also affects how data can be aggregated:

- **Distributive operators:** Operators like SUM, MIN, and MAX allow you to calculate aggregate data from data that is already partially aggregated (e.g., the yearly sum is just the sum of the monthly sums).
- **Non-distributive operators:** Operators like AVG cannot be easily stacked. To calculate an average correctly across aggregated levels, you must use **Support Measures** (like a Count) to keep track of the underlying events.

The most reliable solution is to always **aggregate data directly from the primary view** to ensure accuracy.



Date	StockLevel
1/1/1999	100
10/2/1999	200
30/4/1999	60
5/6/1999	85
18/7/1999	125
31/12/1999	110

4-month period	Stock Level	Count	Stock Level
4/1999	120	3	360
8/1999	105	2	210
12/1999	110	1	110

1999 Average: **113,33**

1999 Average: **111,66** **113,33**

The correct solution is always the one that can be obtained by aggregating data directly from the primary view

1.12 Implementing Materialized Views in Relational Schemata

When you decide to store pre-calculated aggregate data (materialized views), you must choose a relational storage architecture. Each method balances performance, storage requirements, and design complexity.

The Single Fact Table Approach

The simplest method is storing all primary and aggregate data in the main fact table of the star schema.

- **Aggregation Levels:** To identify the aggregation level, NULL values are inserted into the dimension tables for any attributes whose aggregation level is “finer” than the current summary.
- **Drawbacks:** The fact table size increases significantly, which can negatively affect query performance.

SALES	<u>keyS</u>	<u>keyD</u>	<u>keyP</u>	quantity	receipts	
	1	1	1	170	85	
	2	1	1	300	150	
	3	1	1	1700	850	
						

STORE	<u>keyS</u>	store	storeCity	state	
	1	COOP1	Columbus	Ohio	
	2	—	Austin	Texas	
	3	—	—	Texas	
					

Figure 1.8: Row 2 of the SALES table refers to the aggregation pattern {storeCity, date, product}; for this reason, the specific “store” field in the STORE table is null (—). Row 3 of SALES refers to the most aggregated pattern {state, date, product}, resulting in null values for both store and storeCity in the dimension table.

The Constellation Schema Approach

A common alternative is to store different group-by sets in separate fact tables while keeping the original dimension tables merged.

- **Structure:** Aggregated fact tables do not include foreign keys for dimensions that have been completely summarized away.
- **Advantages:** This approach optimizes access because each fact table only contains data at a specific level.

- **Drawbacks:** Dimension tables remain large and still require NULL values for attributes that are invalid at a specific level.

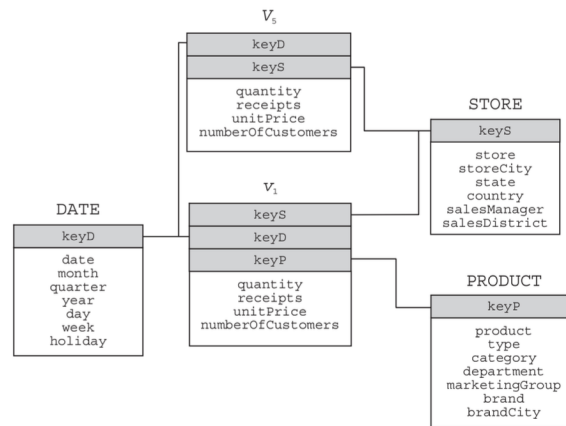


Figure 1.9: Example of a constellation schema where multiple fact tables (V_1 and V_5) represent different aggregation levels. While the fact tables are separated to optimize access, they continue to share the same wide dimension tables (DATE, STORE, PRODUCT).

Replicating Dimension Tables

For maximum speed, you can replicate dimension tables so each view has its own dedicated dimensions.

- **Structure:** Each new dimension table contains only the attributes valid for that specific aggregation level.
- **Drawbacks:** While this offers the best performance, it consumes a large amount of disk space due to redundant data and the need to store multiple versions of the same dimensions.

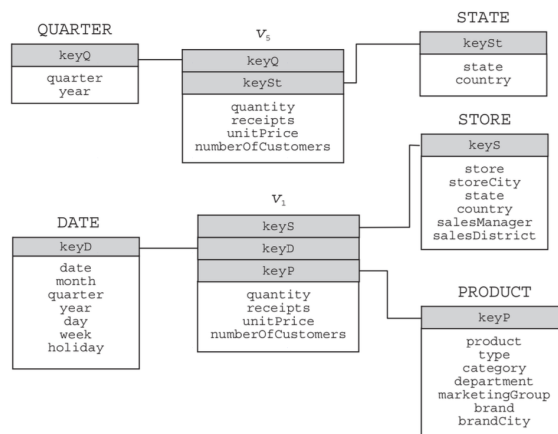


Figure 1.10: Implementation of materialized views through dimension replication. For the aggregated fact table V_5 , dedicated and smaller dimension tables (QUARTER and STATE) are created to maximize query performance at the cost of higher storage consumption.

The Snowflaking Compromise

A balanced solution applies **snowflaking** specifically at the aggregation levels where views are materialized.

- **Structure:** Instead of duplicating tables, dimensions are normalized (split) into hierarchies. The primary fact table connects to the lowest levels, while aggregated fact tables connect directly to higher-level tables.
- **Advantages:** This organizes data by level and avoids full table replication while maintaining optimized join paths.

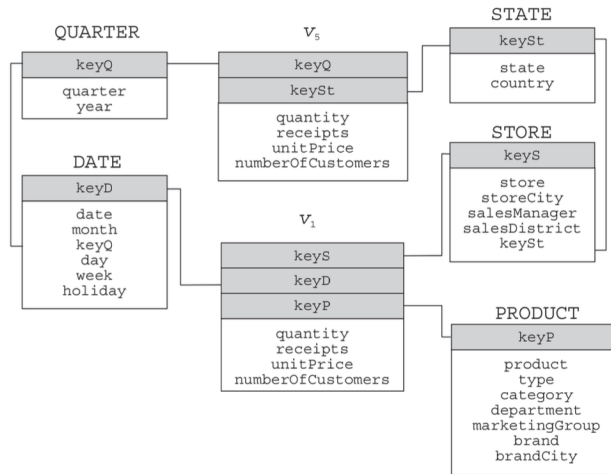


Figure 1.11: A targeted snowflaking approach applied to materialized views. The hierarchies are normalized exactly at the levels required by the aggregate view V_5 (specifically **QUARTER** and **STATE**), creating a balanced structure that avoids dimensional data replication.

1.13 Managing and Routing Aggregate Queries

Using multiple fact tables creates a routing problem: the system must determine which view answers a query with the lowest execution cost.

Aggregate Navigators

This task is performed by the **aggregate navigator**, a software module that automatically intercepts and rewrites OLAP queries to run on the “best” available view.

- **Current Limitations:** Most commercial navigators only manage *distributive* aggregate operators (like SUM or MIN).
- **Impact on Support Measures:** Because these tools struggle with non-distributive operators, the practical usefulness of complex “support measures”, like “Count”, is currently limited in most real-world systems.

1.14 The Logical Design Phase

Logical design is the process of transforming a conceptual model (the Dimensional Fact Model or DFM) into a final logical schema for a data mart. Unlike operational databases (OLTP), logical design for data warehousing relies on techniques like **data redundancy** (storing the same data multiple times to avoid calculations) and **denormalization** (combining tables to avoid joins) to optimize for analytical performance.

The transformation process requires four primary inputs:

- The **Conceptual Schema** (DFM).
- The expected **Workload** (user queries).
- The anticipated **Data Volume**.
- Physical **System Constraints** (hardware and software).

Designers typically follow these steps to define the logical schema:

1. **Choosing the paradigm:** deciding between a Star or Snowflake schema.
2. **Translation:** converting fact schemata into the selected logical model.
3. **Materializing views:** pre-calculating specific aggregations.
4. **Optimization:** implementing performance aids like indexing.

Paradigm Selection: Star vs. Snowflake

Opinions on “snowflaking” (normalizing dimension tables) vary. Some experts believe it contradicts the core philosophy of data warehousing by introducing unnecessary complexity. However, snowflaking is beneficial in certain scenarios:

- **High Cardinality Ratios:** Normalization saves significant storage space when many primary dimension records share a few secondary attributes (e.g., millions of products but only a few product types).
- **Shared Hierarchies:** Secondary dimension tables, such as a geographical **CITY** hierarchy, can be reused across different dimensions like **WAREHOUSE** or **ORDER**.
- **Materialized Aggregate Views:** Secondary dimension tables can be defined at the exact aggregation levels required by aggregate fact tables.

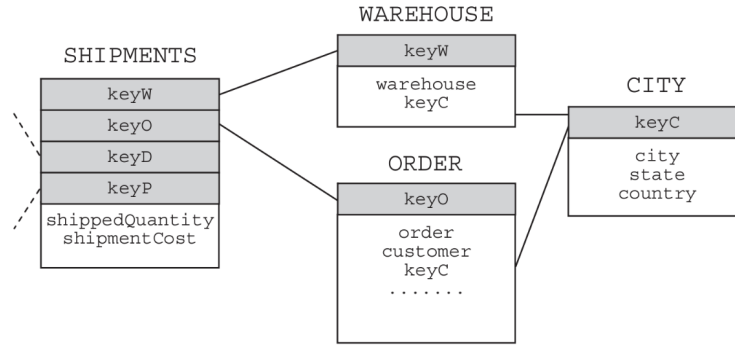


Figure 1.12: Benefit of snowflaking: shared hierarchies. In this scenario, the secondary dimension table **CITY** is shared between different business processes, allowing for cleaner organization and consistency across the data mart.

Converting DFM to Star Schemata

The fundamental rule for creating a Star schema is as follows:

Create a central fact table containing all measures and descriptive attributes directly connected to the fact. Then, create one dimension table for each hierarchy that contains all associated attributes.

Descriptive and Cross-Dimensional Attributes

Logical design must also address attributes that do not participate in aggregation.

Descriptive attributes provide useful context but are not used for sums or averages.

- If connected to a dimensional attribute, it is included in that attribute's dimension table.
- If connected directly to the fact, it is placed in the central fact table. Note that such attributes will be **omitted from aggregated views** because they are only compatible with the primary event granularity.

Cross-dimensional attributes b represent a many-to-many relationship between multiple dimensional attributes $(a_1, \dots, a_m)$. Their logical translation requires a **new table** containing the attribute and the keys of the connected dimensions. Designers must choose between using natural composite keys or generating surrogate keys based on storage space requirements.

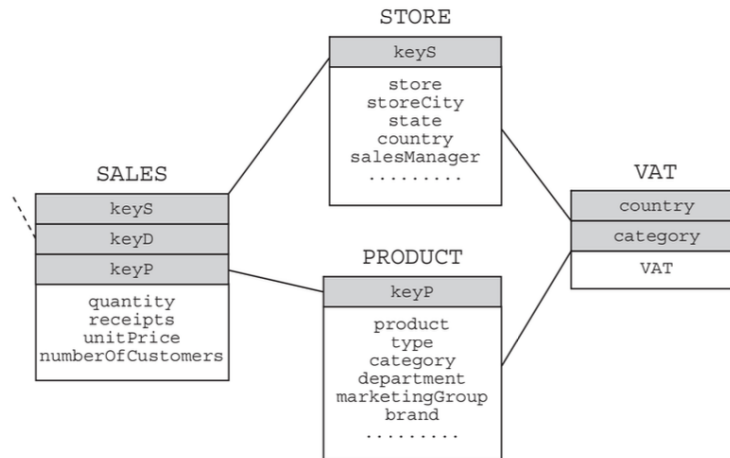


Figure 1.13: Translation of a cross-dimensional attribute: the relationship is handled by creating a separate table. The primary key of this new table is a combination of the keys from the involved dimensions (e.g., `keyS` and `keyP`).

1.15 Managing Shared Hierarchies in Logical Design

In many business processes, the same hierarchy might appear multiple times within a single fact table or across different facts. Instead of creating multiple identical dimension tables, which is inefficient, we use specific logical strategies based on how much information is shared.

Fully Shared Hierarchies

A hierarchy is **fully shared** when two or more dimensions contain the exact same set of attributes. In this case, you do not need to replicate the dimension table. Instead, you simply import the primary key of that single dimension table into the fact table multiple times.

Each occurrence of the key in the fact table represents a different **role** played by that dimension. For example, in a **CALLS** fact table, both the “calling” number and the “called” number share the same structure (phone number, type, district). As shown in the example below, the fact table uses two foreign keys (`keyCalling` and `keyCalled`) that both point to the same **NUMBER** dimension table.

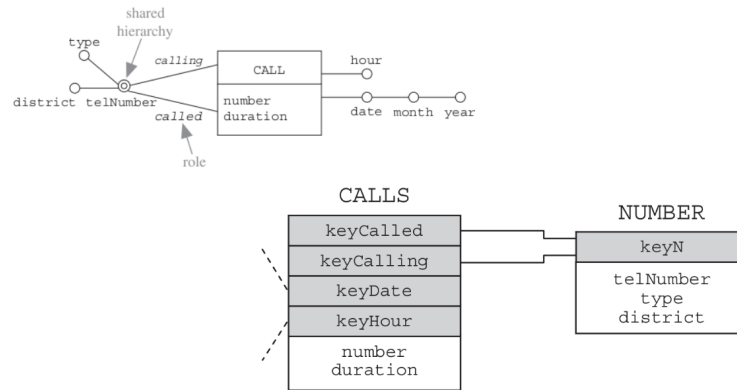


Figure 1.14: Logical translation of a fully shared hierarchy. The **CALLS** fact table references the **NUMBER** dimension twice, using different foreign keys to distinguish between the caller and the receiver roles.

Partially Shared Hierarchies

A hierarchy is **partially shared** when two dimensions link to the same set of higher-level attributes but start from different points. For instance, a **SHIPMENT** fact might track both the *warehouse* and the *order*. While these are different entities, they might both roll up to a shared geographical hierarchy consisting of **city**, **state**, and **country**.

Designers can handle this in two ways:

1. **Replication:** Duplicate the common geographical fields within both the **WAREHOUSE** and **ORDER** dimension tables. This maintains a pure Star Schema but increases data redundancy.
2. **Snowflaking:** Create a third, separate table (e.g., **CITY**) that contains the shared attributes. Both the **WAREHOUSE** and **ORDER** tables will then include a foreign key (**keyC**) to reference this shared table.

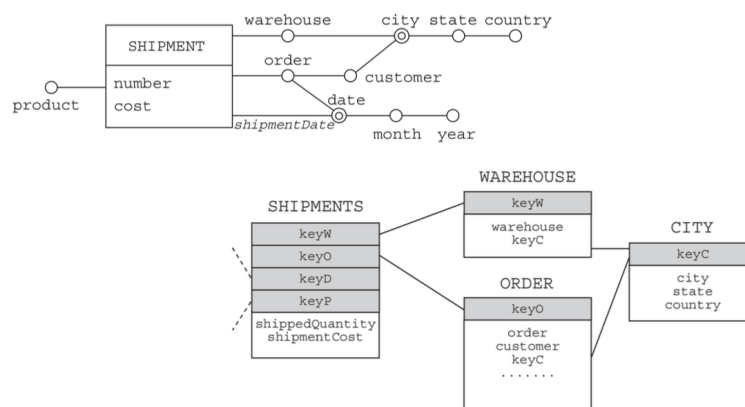


Figure 1.15: Logical translation of a partially shared hierarchy. By using a snowflake approach, the common geographical attributes are moved to a separate **CITY** table, which is then reused by both the **WAREHOUSE** and **ORDER** dimensions.

1.16 Managing Many-to-Many Relationships (Multiple Arcs)

In the Dimensional Fact Model (DFM), many-to-many relationships are represented by **multiple arcs**. A typical example is a book having more than one author. Because the standard Star Schema is designed for one-to-many relationships, we must use specific logical design strategies to handle these cases.

The Bridge Table Approach

The most common solution is to insert an additional table called a **bridge table**. This table maps the relationship between the two entities.

- **Structure:** The primary key of the bridge table is a combination of the keys from the attributes connected by the multiple arc (e.g., **keyB** from Book and **keyA** from Author).
- **Weights:** An optional **weight** attribute can be added to assign a percentage of relevance to each record. For example, if two authors wrote a book, each might be assigned a 0.5 weight.

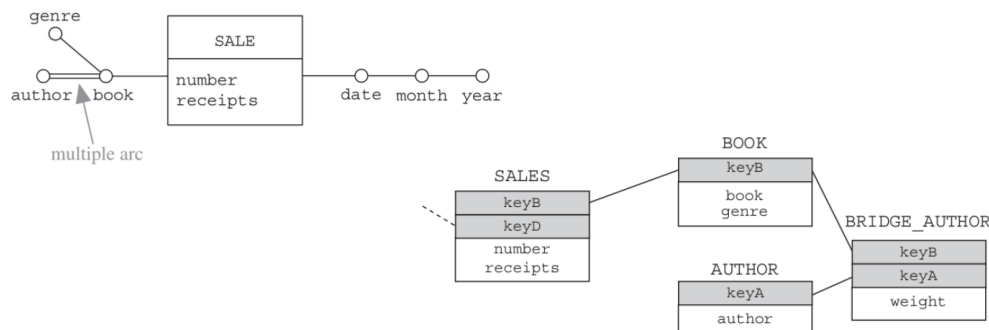


Figure 1.16: Logical translation using a bridge table. The **BRIDGE_AUTHOR** table connects books to multiple authors without duplicating the primary sales event.

This structure allows for two types of OLAP queries:

1. **Weighted Queries:** These multiply the measure (e.g., receipts) by the weight to show the actual total for each individual (e.g., how much revenue an author truly earned).

```
SELECT AUTHOR.author, sum(SALES.receipts * BRIDGE_AUTHOR.weight)
FROM AUTHOR, BRIDGE_AUTHOR, BOOK, SALES
WHERE AUTHOR.keyA = BRIDGE_AUTHOR.keyA
AND BRIDGE_AUTHOR.keyB = BOOK.keyB
AND BOOK.keyB = SALES.keyB
GROUP BY AUTHOR.author
```

2. **Impact Queries:** These ignore the weight. The resulting totals are higher because the full measure is attributed to every person involved.

```
SELECT AUTHOR.author, sum(SALES.number)
FROM AUTHOR, BRIDGE_AUTHOR, BOOK, SALES
WHERE AUTHOR.keyA = BRIDGE_AUTHOR.keyA
AND BRIDGE_AUTHOR.keyB = BOOK.keyB
AND BOOK.keyB = SALES.keyB
GROUP BY AUTHOR.author
```

The Push-down Solution

If you want to stick strictly to a Star Schema structure, you can use the **push-down** method. This involves making the **granularity** of the fact table finer so the multiple arc is modeled directly in the fact table.

- **Method:** A new dimension (**keyA**) is added to the fact table.
- **Effect:** A single business event is split into multiple rows. If a book has two authors, the sale of that book appears twice in the fact table, once for each author.

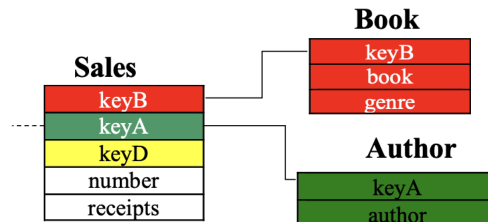


Figure 1.17: Push-down implementation: the **Author** dimension is moved into the **SALES** fact table, increasing its granularity.

Comparison of Strategies

While both solutions provide the same information, they have different operational impacts:

Feature	Push-down	Bridge Table
Redundancy	High (replicates fact rows)	Low (fact table stays small)
Complexity	Weight logic is hardcoded	Weight logic is dynamic
Performance	Faster queries (less joins)	More joins required
Calculation	During data loading (ETL)	During the query execution

Table 1.2: Comparison between Push-down and Bridge Table solutions.

1.17 Handling Degenerate and Junk Dimensions

In logical design, we must decide how to represent hierarchies that consist of only a single attribute. These are known as **degenerate dimensions**. While we usually create separate dimension tables for hierarchies, these special cases require a different approach to ensure query efficiency and save storage space.

Degenerate Dimensions

A **degenerate dimension** is a hierarchy that contains only one attribute. Standard practice suggests two ways to implement them:

- **Dedicated Dimension Table:** If the single attribute is very long (significantly larger than a standard surrogate key), it is reasonable to create a separate dimension table for it.
- **Direct Storage in Fact Table:** Most often, degenerate dimensions are stored directly in the fact table. This reduces the total number of joins needed for a query, which improves performance.

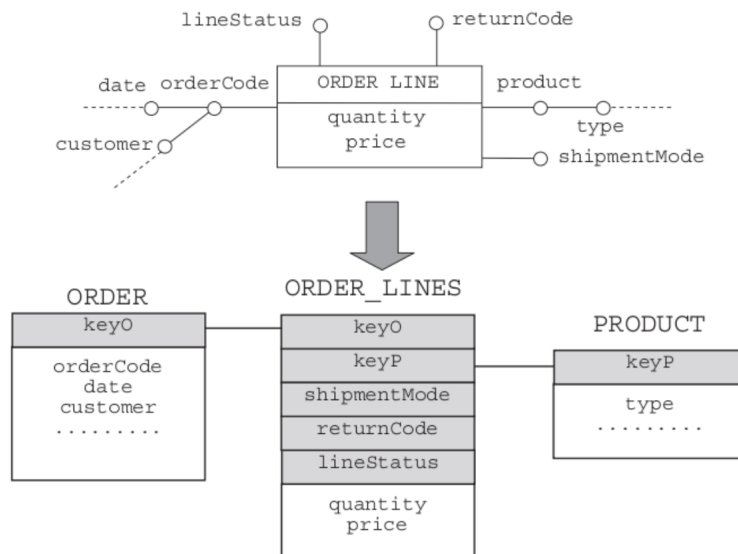


Figure 1.18: Translation of degenerate dimensions directly into the fact table. In this **ORDER_LINES** example, attributes like *lineStatus*, *returnCode*, and *shipmentMode* are stored as part of the fact table rather than having their own separate dimension tables.

Junk Dimensions

Junk dimensions offer an alternative solution for handling multiple degenerate dimensions. A junk dimension is a single dimension table that combines a set of different degenerate dimensions into one place.

Characteristics of junk dimensions include:

- **No Functional Dependencies:** There are no relationships between the attributes combined in the table.
- **All Combinations are Valid:** Because there are no dependencies, every possible combination of the involved attributes can exist.
- **Size Constraints:** This solution is only practical when the total number of valid attribute combinations is relatively small. If there are too many combinations, the junk dimension table will become too large to be useful.

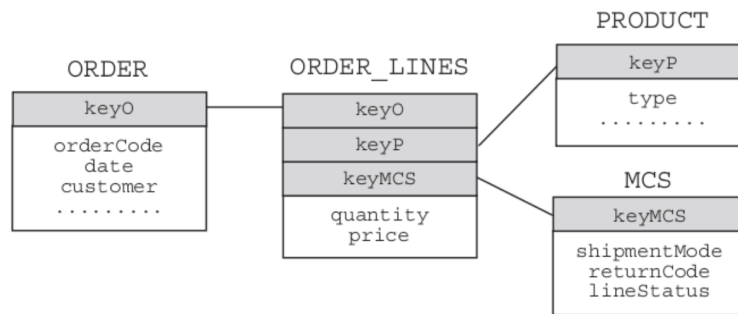


Figure 1.19: Implementation of a Junk Dimension (MCS). Here, `shipmentMode`, `returnCode`, and `lineStatus` are moved out of the fact table and combined into a single dimension table called MCS to keep the central table cleaner.